# Deep Learning para visión de computadoras

## Lab 1.3 Arquitecturas CNN de referencia: LeNet-5, AlexNet, GoogLeNet y VGG16

**Unidad 1.** Fundamentos de visión por computadora y redes convolucionales **1.3** Arquitecturas CNN para clasificación




Esta práctica toma como referencia *"Arquitecturas de Redes Convolucionales"* (LeNet-5, AlexNet, GoogLeNet, VGG16), parte A definicion original de los modelos y parte B entrenamiento y evalúación sobre CIFAR-10, para poder comparar capacidad, número de parámetros, tiempo de entrenamiento y desempeño real.

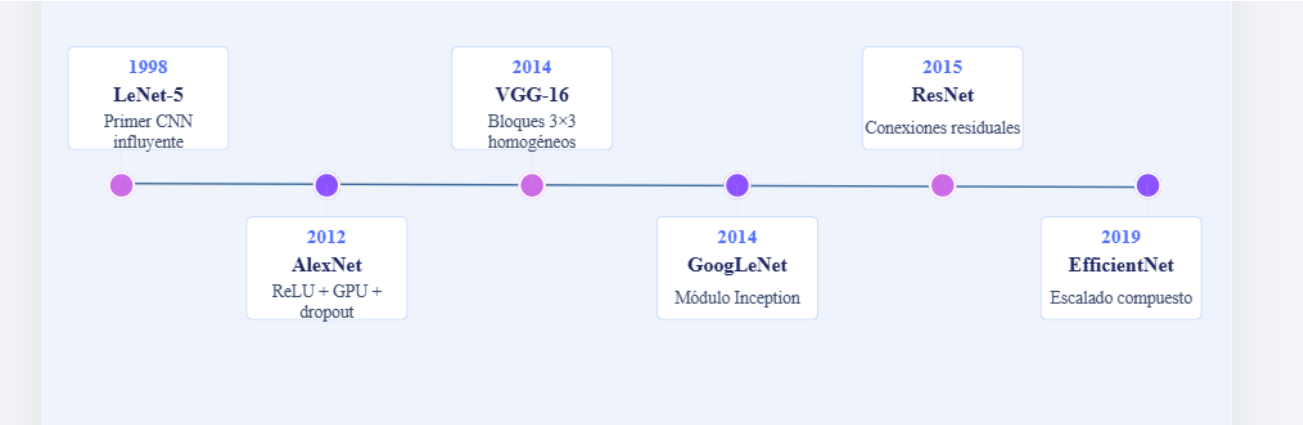
**Objetivos de la práctica**

1. Implementar en PyTorch cuatro arquitecturas CNN clásicas adaptadas a CIFAR-10.
2. Entrenar y evaluar cada una bajo las mismas condiciones (mismos datos, épocas, optimizador).
3. Comparar número de parámetros, tiempo de entrenamiento y accuracy de test.
4. Discutir la relación entre profundidad/capacidad del modelo y su desempeño/eficiencia.

# PARTE A

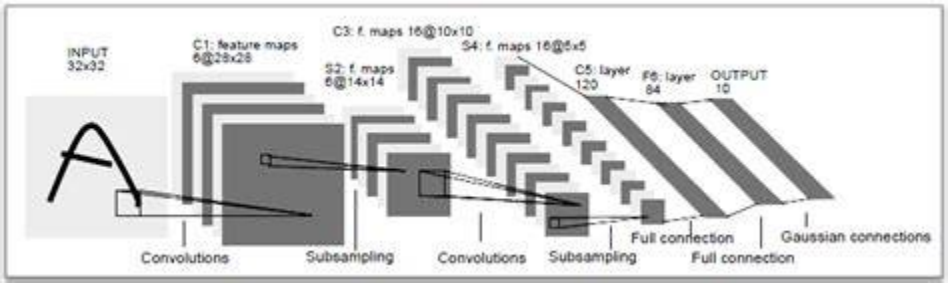
## Arquitecturas de Redes Convolucionales

En el campo del deep learning y computer vision, una de estas competiciones, que hoy en día ya es un benchmark establecido, es la clasificación de imágenes en el dataset Imagenet. En 2012, la primera red neuronal convolucional entró en esta competición consiguiendo una mejora de 10 puntos sobre la mejor solución hasta la fecha, lo cual supuso un detonante para el campo del deep learning.




# LeNet-5

| Model | Year | Accuracy metrics (Top-1/Top-5) | Parameters (Million) | GFLOPs | Computational costs (MACs/Inference Time) | Key features | Improvements | Advantages | Disadvantages | Suitable applications |
|---|---|---|---|---|---|---|---|---|---|---|
| LeNet-5 [11] | 1998 | ~ 99.0%s/ ~ 99.05%<br><br>(MNIST) | 0.06 | 0.002 | ~ 0.001B MACs/ < 1 ms | 5 layers, small filters, pooling | First practical CNN | Simple, fast training | Limited to small datasets | Digit recognition, basic classification |




Esta `CNN` es alimentada por imágenes de 32x32 píxeles de dígitos manuscritos, y está formada por tres capas convolucionales, dos de las cuales reducen la dimensionalidad de los mapas de características mediante *average pooling*. La salida de la última capa convolucional es conectada un perceptrón multicapa de dos capas, que tienen la responsabilidad de dar la predicción final. Artículo de publicación original . A continuación puedes ver un ejemplo de implementación en `Pytorch` .

```
import torch

def block(c_in, c_out, k=3, p=1, s=1):
    return torch.nn.Sequential(
        torch.nn.Conv2d(c_in, c_out, k, padding=p, stride=s),
        torch.nn.Tanh(),
        torch.nn.AvgPool2d(2, stride=2)
    )

def block2(c_in, c_out):
    return torch.nn.Sequential(
        torch.nn.Linear(c_in, c_out),
        torch.nn.ReLU()
    )

class LeNet5(torch.nn.Module):
  def __init__(self, n_channels=1, n_outputs=10):
```

```
    super().__init__()
    #self.pad = torch.nn.ConstantPad2d(2, 0.)
    self.conv1 = block(n_channels, 6, 5, p=0)
    self.conv2 = block(6, 16, 5, p=0)
    self.conv3 = torch.nn.Sequential(
        torch.nn.Conv2d(16, 120, 5, padding=0),
        torch.nn.Tanh()
    )
    self.fc1 = block2(120, 84)
    self.fc2 = torch.nn.Linear(84, 10)

  def forward(self, x):
    #x = self.pad(x)
    x = self.conv1(x)
    x = self.conv2(x)
    x = self.conv3(x)
    x = x.view(x.shape[0], -1)
    x = self.fc1(x)
    x = self.fc2(x)
    return x
```

```
# Rastreo de dimensiones para LeNet-5 con entrada (64, 1, 32, 32)
lenet5 = LeNet5()  # Creamos la instancia del modelo
x = torch.randn(64, 1, 32, 32)

# Capa a capa (la implementación agrupa conv + pool en bloques)
print("Entrada:                          ", x.shape)
x = lenet5.conv1(x)
print("Bloque Conv1 + AvgPool1 (6@5x5):  ", x.shape)
x = lenet5.conv2(x)
print("Bloque Conv2 + AvgPool2 (16@5x5): ", x.shape)
x = lenet5.conv3(x)
print("Conv3 (120@5x5):                  ", x.shape)
x = x.view(x.size(0), -1)
print("Flatten:                          ", x.shape)
x = lenet5.fc1(x)
print("FC1 (84 neuronas):                ", x.shape)
x = lenet5.fc2(x)
print("FC2 / Salida (10 clases):         ", x.shape)
```

```
Entrada:                          torch.Size([64, 1, 32, 32])
Bloque Conv1 + AvgPool1 (6@5x5):  torch.Size([64, 6, 14, 14])
Bloque Conv2 + AvgPool2 (16@5x5): torch.Size([64, 16, 5, 5])
Conv3 (120@5x5):                  torch.Size([64, 120, 1, 1])
Flatten:                          torch.Size([64, 120])
FC1 (84 neuronas):                torch.Size([64, 84])
FC2 / Salida (10 clases):         torch.Size([64, 10])
```

# Actividad

## Ejercicio: Rastreo de dimensiones en LeNet-5

Dada la ejecución del modelo con un lote de entrada aleatorio:

CODE lenet5 = LeNet5() output = lenet5(torch.randn(64, 1, 32, 32)) output.shape

Completa # y ?

Entrada: ( ? , ? , ? , ? )

Conv1 (# filtros de #x# (kernel), stride=#, padding=#): (64, ?, ?, ?)

AvgPool / MaxPool 1 (filtro #x#, stride=#): (64, ?, ?, ?)

Conv2 (# filtros de #x#, stride=#, padding=0): (64, ?, ?, ?)

AvgPool / MaxPool 2 (filtro #x#, stride=#): (64, ?, ?, ?)

Aplanamiento (Flatten): (64, ?)

FC1 / Densa 1 (# neuronas): (64, ?)

FC2 / Densa 2 (# neuronas): (64, ?)

Salida final (FC3 / # clases): torch.Size([?, ?])

Demuestra tus resultados con la formula : $O = \left\lfloor \frac{N + 2P - F}{S} \right\rfloor + 1$

```
lenet5 = LeNet5()
output = lenet5(torch.randn(64, 1, 32, 32))
output.shape
```

```
torch.Size([64, 10])
```

# Respuesta — Dimensiones de LeNet-5

Con entrada `(64, 1, 32, 32)` y usando la fórmula `O = floor((N + 2P − F) / S) + 1`:

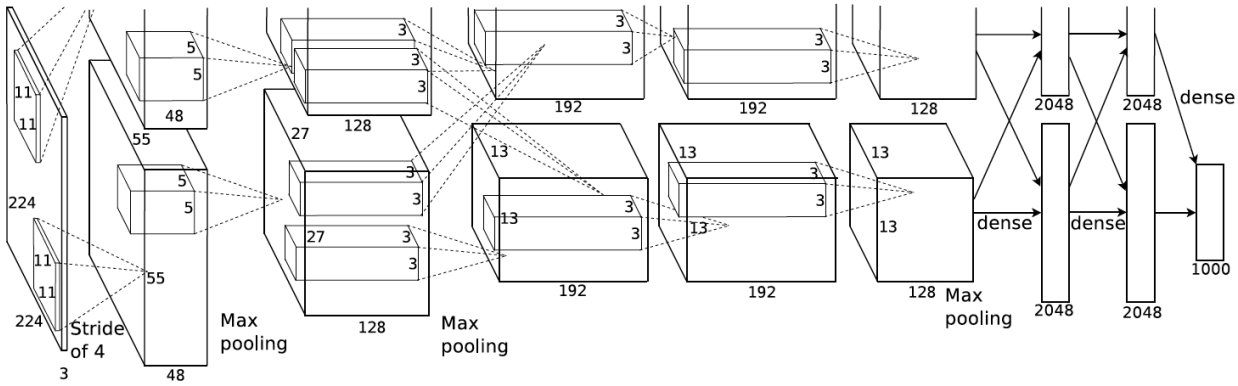
- **Entrada:** `(64, 1, 32, 32)`
- **Bloque Conv1 + AvgPool1** (6 filtros de 5×5, stride=1, padding=0; luego pool 2×2, stride=2): `(64, 6, 14, 14)` → conv: `floor((32 − 5)/1) + 1 = 28`; pool: `floor(28/2) = 14`
- **Bloque Conv2 + AvgPool2** (16 filtros de 5×5, stride=1, padding=0; luego pool 2×2, stride=2): `(64, 16, 5, 5)` → conv: `floor((14 − 5)/1) + 1 = 10`; pool: `floor(10/2) = 5`
- **Conv3** (120 filtros de 5×5, stride=1, padding=0): `(64, 120, 1, 1)` → `floor((5 − 5)/1) + 1 = 1`
- **Flatten:** `(64, 120)` → `120 × 1 × 1 = 120`
- **FC1** (84 neuronas): `(64, 84)`
- **FC2** / **Salida** (10 clases): `torch.Size([64, 10])`

> Nota: la implementación mostrada agrupa cada convolución con su pooling en un bloque `Sequential`. La versión adaptada de más abajo usa `padding=2` en la primera convolución para conservar 32×32.

# AlexNet

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| AlexNet [4] | 2012 | 57.2%/84.6% (ImageNet) | 60 | 1.43 | ~ 0.72B MACs/ ~ 8.9 ms | ReLU activation, Dropout, deeper layers | GPU usage, improved accuracy | High accuracy for ImageNet | Overfitting risk, computational cost | Image classification, object detection |

AlexNet ganó la competición de Imagenet en 2012. Es una arquitectura similar a LeNet5 pero más profunda. Fue la primera red convolucional que ganó la competición, y también la primera en ser entrenada en GPUs.




Implementadas en `Pytorch` podemos descargar redes convolucionales con el paquete `torchvision` (en varios paquetes) .

```
import torchvision
alexnet = torchvision.models.AlexNet()
alexnet
```

```
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,
ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)
```

# Actividad

Analiza la siguiente celda de ejecución correspondiente a la arquitectura clásica AlexNet: cual es el lote de imagenes de entrada canales, dimensiones? y porque su salida es el 1000 indica las lineas de codigo y que representa este numero?

```
output = alexnet(torch.randn(64, 3, 224, 224))
output.shape
```

```
torch.Size([64, 1000])
```

# Respuesta — AlexNet

La ejecución `output = alexnet(torch.randn(64, 3, 224, 224))` indica:

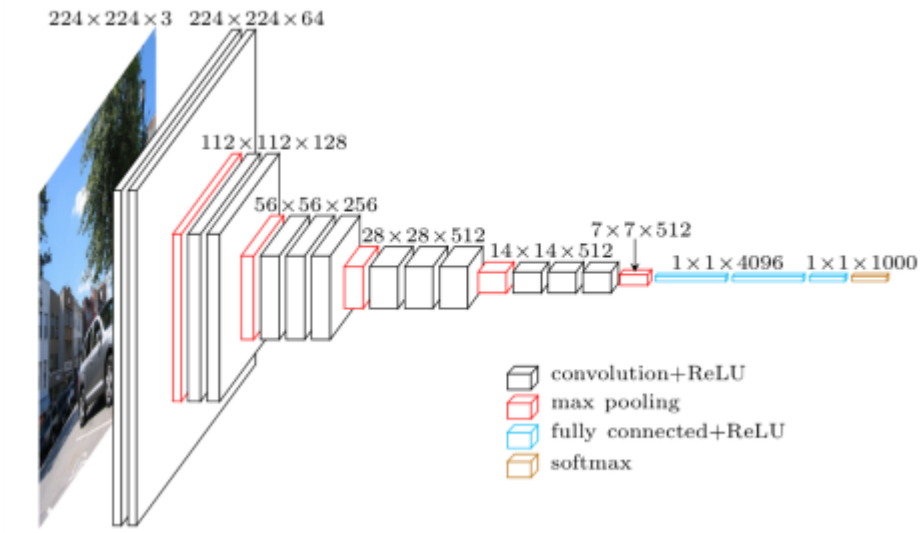
- **Lote de imágenes de entrada:** 64 imágenes.
- **Canales:** 3 (RGB).
- **Dimensiones espaciales:** 224×224 píxeles (tamaño estándar de ImageNet).

La salida es `(64, 1000)` porque la última capa lineal de AlexNet ( `classifier[6]` ) tiene **1000 neuronas de salida**, una por cada clase de ImageNet. La línea clave es `torch.nn.Linear(4096, num_classes)` con `num_classes=1000` en la definición original de `torchvision.models.alexnet` .

# VGG

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| VGG [5, 80] | 2014 | 71.5%/<br><br>92.7% (ImageNet) | 138 | 15.47 | ~ 15.5B MACs/ ~ 15.6 ms | 16–19 layers, small filters | Improved feature extraction | Simple structure, high accuracy | Large model size, slow training | Image classification, transfer learning |

Si bien VGG no ganó, tuvo mucho éxito ya que sentó un precedente en el patrón del diseño de las redes convolucionales. Este patrón consiste en usar siempre capas convolucionales con filtros de 3x3, *stride* y *padding* 1 (manteniendo el tamaño de las entradas constante) y usar *max pool* para reducir a la mitad los mapas de características. Además, después de cada capa se dobla el número de filtros.




```
vgg16 = torchvision.models.vgg16()
vgg16
```

```
VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU(inplace=True)
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (5): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (6): ReLU(inplace=True)
    (7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (8): ReLU(inplace=True)
    (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (13): ReLU(inplace=True)
    (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (15): ReLU(inplace=True)
    (16): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (17): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (18): ReLU(inplace=True)
    (19): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (20): ReLU(inplace=True)
    (21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (22): ReLU(inplace=True)
    (23): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (24): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (25): ReLU(inplace=True)
    (26): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (27): ReLU(inplace=True)
    (28): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (29): ReLU(inplace=True)
    (30): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(7, 7))
  (classifier): Sequential(
    (0): Linear(in_features=25088, out_features=4096, bias=True)
    (1): ReLU(inplace=True)
    (2): Dropout(p=0.5, inplace=False)
    (3): Linear(in_features=4096, out_features=4096, bias=True)
    (4): ReLU(inplace=True)
    (5): Dropout(p=0.5, inplace=False)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)
```

```
output = vgg16(torch.randn(64, 3, 224, 224))
output.shape
```

```
torch.Size([64, 1000])
```

# Respuesta — Modelos VGG en torchvision

En `torchvision.models.vgg` hay 8 modelos predefinidos (con y sin pesos pre-entrenados):

- `vgg11` y `vgg11_bn`
- `vgg13` y `vgg13_bn`
- `vgg16` y `vgg16_bn`
- `vgg19` y `vgg19_bn`

La diferencia entre ambos es que los que terminan en `_bn` incluyen **Batch Normalization** después de cada capa convolucional.

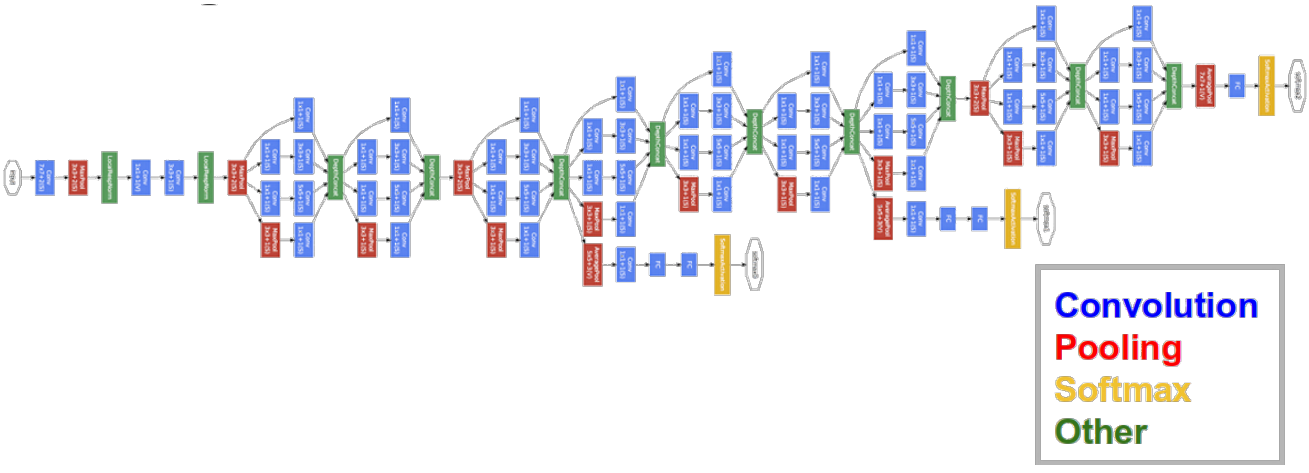
# Actividad modelos VGG

Investiga y define en codigo otros modelos de torchvision.models.vgg cuantos hay ya definidos?

# GoogLeNet

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| GoogleNet [81] | 2014 | 69.8%/<br><br>93.3% (ImageNet) | 7 | 1.5 | ~1.43B MACs/ ~ 6.6 ms | Inception modules, 22 layers | Efficient architecture | Faster, smaller size | Complex to implement | Object recognition, real-time tasks |

Ganadora de la edición de 2014, GoogLeNet es una red mucho más profunda (podemos empezar a ver la tendencia de que cuantas más capas, mejor resultados). Sus desarrolladores implementaron los módulos *inception*, los cuales aplican diferentes capas convolucionales, con diferentes tamaños de filtros, en paralelo sobre las mismas entradas y luego concatenan las salidas. Además, para poder entrenar esta red tan profunda, se llevan a cabo predicciones en diversas capas intermedias para poder tener gradientes fluyendo hacia las capas iniciales.




```
googlenet = torchvision.models.GoogLeNet()
googlenet
```

```
/tmp/opencode/venv_run/lib/python3.14/site-packages/torchvision/models/
googlenet.py:47: FutureWarning: The default weight initialization of GoogleNet will
be changed in future releases of torchvision. If you wish to keep the old behavior
(which leads to long initialization times due to scipy/scipy#11299), please set
init_weights=True.
  warnings.warn(
```

```
GoogLeNet(
  (conv1): BasicConv2d(
    (conv): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
    (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
  )
  (maxpool1): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,
ceil_mode=True)
  (conv2): BasicConv2d(
    (conv): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
  )
  (conv3): BasicConv2d(
    (conv): Conv2d(64, 192, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
  )
  (maxpool2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,
ceil_mode=True)
  (inception3a): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(192, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(192, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(96, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(96, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(192, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(16, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1,
ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(192, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, bias=True,
```

```
track_running_stats=True)
      )
    )
  )
  (inception3b): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(128, 192, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(32, 96, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(96, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1,
ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
  )
  (maxpool3): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,
ceil_mode=True)
  (inception4a): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(480, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(480, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(96, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
```

```
      (1): BasicConv2d(
        (conv): Conv2d(96, 208, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(208, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(480, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(16, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(16, 48, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(48, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1,
ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(480, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
  )
  (inception4b): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(512, 160, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(160, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(512, 112, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(112, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(112, 224, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(224, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(512, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(24, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(24, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
```

```
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1,
ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(512, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
  )
  (inception4c): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(512, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(24, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(24, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1,
ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(512, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
  )
  (inception4d): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(512, 112, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(112, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
```

```
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(512, 144, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(144, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(144, 288, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(288, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(512, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1,
ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(512, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
  )
  (inception4e): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(528, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(528, 160, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(160, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(160, 320, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(320, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(528, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
```

```
      )
      (1): BasicConv2d(
        (conv): Conv2d(32, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1,
ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(528, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
  )
  (maxpool4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=True)
  (inception5a): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(832, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(256, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(832, 160, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(160, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(160, 320, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(320, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(832, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(32, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(32, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1,
ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(832, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
```

```
    )
  )
  (inception5b): Inception(
    (branch1): BasicConv2d(
      (conv): Conv2d(832, 384, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(384, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
    (branch2): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(832, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(192, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(384, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch3): Sequential(
      (0): BasicConv2d(
        (conv): Conv2d(832, 48, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(48, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
      (1): BasicConv2d(
        (conv): Conv2d(48, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
    (branch4): Sequential(
      (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1,
ceil_mode=True)
      (1): BasicConv2d(
        (conv): Conv2d(832, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
      )
    )
  )
  (aux1): InceptionAux(
    (conv): BasicConv2d(
      (conv): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
    (fc1): Linear(in_features=2048, out_features=1024, bias=True)
    (fc2): Linear(in_features=1024, out_features=1000, bias=True)
    (dropout): Dropout(p=0.7, inplace=False)
  )
  (aux2): InceptionAux(
    (conv): BasicConv2d(
      (conv): Conv2d(528, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
    )
```

```
    (fc1): Linear(in_features=2048, out_features=1024, bias=True)
    (fc2): Linear(in_features=1024, out_features=1000, bias=True)
    (dropout): Dropout(p=0.7, inplace=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (dropout): Dropout(p=0.2, inplace=False)
  (fc): Linear(in_features=1024, out_features=1000, bias=True)
)
```

```
output = googlenet(torch.randn(64, 3, 224, 224))
output[0].shape
```

```
torch.Size([64, 1000])
```

# Respuesta — GoogLeNet

1. **Bloques Inception:** la arquitectura original tiene **9 bloques Inception** organizados en 3 grupos llamados `3a/3b`, `4a/4b/4c/4d/4e` y `5a/5b`.
2. **Ramas paralelas:** cada bloque Inception tiene **4 ramas convolucionales en paralelo**:
   - 1×1 directa.
   - 1×1 → 3×3.
   - 1×1 → 5×5.
   - MaxPool 3×3 → 1×1.
3. **Reemplazo del Flatten:** GoogLeNet usa una capa de **Average Pooling Global** (`AdaptiveAvgPool2d(1)`) al final en lugar de aplanar todo el volumen. Esto reduce drásticamente el número de parámetros porque la capa densa siguiente recibe solo un valor por canal, en vez de todos los píxeles del mapa de características.

# Actividad

¿Cuántos bloques Inception contiene en total la arquitectura y cómo están agrupados por letras (por ejemplo, a, b, etc.)? ¿Cuántas ramas convolucionales paralelas procesan la entrada simultáneamente dentro de ese único bloque? Agrega una imagen que lo ejemplifique ¿qué capa de pooling se utiliza en lugar del tradicional aplanamiento masivo (Flatten) que usaban redes como AlexNet o VGG? ¿Qué ventaja aporta esto al número total de parámetros del modelo?

# PARTE B

**Ajustes respecto a las arquitecturas originales:** las versiones originales de AlexNet y VGG16 fueron diseñadas para imágenes de 224×224 (ImageNet). Entrenarlas tal cual sobre CIFAR-10 (32×32) o reescalando cada imagen a 224×224 sería demasiado costoso. Por eso cada arquitectura se **adapta** aquí para trabajar directamente sobre 32×32 px, manteniendo su estructura conceptual (número y tipo de bloques, filtros, profundidad relativa) pero con canales/strides ajustados. Esto es intencional: la comparación de "capacidad versus eficiencia"

> 💡El dataset Imagenet contiene imágenes comunes descargadas de internet, y el objetivo es el de clasificar imágenes entre 1000 clases diferentes. Es por este motivo que verás que los diferentes modelos en `torchvision` tienen una última capa lineal con 1000 salidas.

# 1. Configuración e importaciones

```
import time
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, random_split
import matplotlib.pyplot as plt
import pandas as pd

torch.manual_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Dispositivo: {device}")
```

```
Dispositivo: cuda
```

# 2. Carga y preprocesamiento de CIFAR-10

Se usa CIFAR-10 (60,000 imágenes de 32×32 px a color, 10 clases) tal como indica el programa de la asignatura. Se separa un subconjunto de validación a partir del set de entrenamiento.

```
transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)),
])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)),
])

full_train_set = torchvision.datasets.CIFAR10(root="./data", train=True, download=T
test_set = torchvision.datasets.CIFAR10(root="./data", train=False, download=True,

val_fraction = 0.1
val_size = int(len(full_train_set) * val_fraction)
train_size = len(full_train_set) - val_size
train_set, val_set = random_split(full_train_set, [train_size, val_size], generator

BATCH_SIZE = 128
train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True, num_worke
val_loader = DataLoader(val_set, batch_size=BATCH_SIZE, shuffle=False, num_workers=
test_loader = DataLoader(test_set, batch_size=BATCH_SIZE, shuffle=False, num_worker
```

```
classes = ("avión", "automóvil", "pájaro", "gato", "venado", "perro", "rana", "caba
print(f"Train: {len(train_set)} | Val: {len(val_set)} | Test: {len(test_set)}")
```

```
0.0%
0.0%
0.1%
0.1%
0.1%
0.1%
0.1%
0.2%
0.2%
0.2%
0.2%
0.2%
0.2%
0.3%
0.3%
0.3%
0.3%
0.3%
0.4%
0.4%
0.4%
0.4%
0.4%
0.5%
0.5%
0.5%
0.5%
0.5%
0.6%
0.6%
0.6%
0.6%
0.6%
0.7%
0.7%
0.7%
0.7%
0.7%
0.7%
0.8%
0.8%
0.8%
0.8%
0.8%
```

0.9%
0.9%
0.9%
0.9%
0.9%
1.0%
1.0%
1.0%
1.0%
1.0%
1.1%
1.1%
1.1%
1.1%
1.1%
1.2%
1.2%
1.2%
1.2%
1.2%
1.2%
1.3%
1.3%
1.3%
1.3%
1.3%
1.4%
1.4%
1.4%
1.4%
1.4%
1.5%
1.5%
1.5%
1.5%
1.5%
1.6%
1.6%
1.6%
1.6%
1.6%
1.7%
1.7%
1.7%
1.7%
1.7%

1.7%
1.8%
1.8%
1.8%
1.8%
1.8%
1.9%
1.9%
1.9%
1.9%
1.9%
2.0%
2.0%
2.0%
2.0%
2.0%
2.1%
2.1%
2.1%
2.1%
2.1%
2.2%
2.2%
2.2%
2.2%
2.2%
2.2%
2.3%
2.3%
2.3%
2.3%
2.3%
2.4%
2.4%
2.4%
2.4%
2.4%
2.5%
2.5%
2.5%
2.5%
2.5%
2.6%
2.6%
2.6%
2.6%

2.6%
2.7%
2.7%
2.7%
2.7%
2.7%
2.7%
2.8%
2.8%
2.8%
2.8%
2.8%
2.9%
2.9%
2.9%
2.9%
2.9%
3.0%
3.0%
3.0%
3.0%
3.0%
3.1%
3.1%
3.1%
3.1%
3.1%
3.2%
3.2%
3.2%
3.2%
3.2%
3.2%
3.3%
3.3%
3.3%
3.3%
3.3%
3.4%
3.4%
3.4%
3.4%
3.4%
3.5%
3.5%
3.5%

3.5%
3.5%
3.6%
3.6%
3.6%
3.6%
3.6%
3.7%
3.7%
3.7%
3.7%
3.7%
3.7%
3.8%
3.8%
3.8%
3.8%
3.8%
3.9%
3.9%
3.9%
3.9%
3.9%
4.0%
4.0%
4.0%
4.0%
4.0%
4.1%
4.1%
4.1%
4.1%
4.1%
4.2%
4.2%
4.2%
4.2%
4.2%
4.2%
4.3%
4.3%
4.3%
4.3%
4.3%
4.4%
4.4%

4.4%
4.4%
4.4%
4.5%
4.5%
4.5%
4.5%
4.5%
4.6%
4.6%
4.6%
4.6%
4.6%
4.7%
4.7%
4.7%
4.7%
4.7%
4.7%
4.8%
4.8%
4.8%
4.8%
4.8%
4.9%
4.9%
4.9%
4.9%
4.9%
5.0%
5.0%
5.0%
5.0%
5.0%
5.1%
5.1%
5.1%
5.1%
5.1%
5.2%
5.2%
5.2%
5.2%
5.2%
5.2%
5.3%

5.3%
5.3%
5.3%
5.3%
5.4%
5.4%
5.4%
5.4%
5.4%
5.5%
5.5%
5.5%
5.5%
5.5%
5.6%
5.6%
5.6%
5.6%
5.6%
5.7%
5.7%
5.7%
5.7%
5.7%
5.7%
5.8%
5.8%
5.8%
5.8%
5.8%
5.9%
5.9%
5.9%
5.9%
5.9%
6.0%
6.0%
6.0%
6.0%
6.0%
6.1%
6.1%
6.1%
6.1%
6.1%
6.2%

6.2%
6.2%
6.2%
6.2%
6.2%
6.3%
6.3%
6.3%
6.3%
6.3%
6.4%
6.4%
6.4%
6.4%
6.4%
6.5%
6.5%
6.5%
6.5%
6.5%
6.6%
6.6%
6.6%
6.6%
6.6%
6.6%
6.7%
6.7%
6.7%
6.7%
6.7%
6.8%
6.8%
6.8%
6.8%
6.8%
6.9%
6.9%
6.9%
6.9%
6.9%
7.0%
7.0%
7.0%
7.0%
7.0%

7.1%
7.1%
7.1%
7.1%
7.1%
7.1%
7.2%
7.2%
7.2%
7.2%
7.2%
7.3%
7.3%
7.3%
7.3%
7.3%
7.4%
7.4%
7.4%
7.4%
7.4%
7.5%
7.5%
7.5%
7.5%
7.5%
7.6%
7.6%
7.6%
7.6%
7.6%
7.6%
7.7%
7.7%
7.7%
7.7%
7.7%
7.8%
7.8%
7.8%
7.8%
7.8%
7.9%
7.9%
7.9%
7.9%

7.9%
8.0%
8.0%
8.0%
8.0%
8.0%
8.1%
8.1%
8.1%
8.1%
8.1%
8.1%
8.2%
8.2%
8.2%
8.2%
8.2%
8.3%
8.3%
8.3%
8.3%
8.3%
8.4%
8.4%
8.4%
8.4%
8.4%
8.5%
8.5%
8.5%
8.5%
8.5%
8.6%
8.6%
8.6%
8.6%
8.6%
8.6%
8.7%
8.7%
8.7%
8.7%
8.7%
8.8%
8.8%
8.8%

8.8%
8.8%
8.9%
8.9%
8.9%
8.9%
8.9%
9.0%
9.0%
9.0%
9.0%
9.0%
9.1%
9.1%
9.1%
9.1%
9.1%
9.1%
9.2%
9.2%
9.2%
9.2%
9.2%
9.3%
9.3%
9.3%
9.3%
9.3%
9.4%
9.4%
9.4%
9.4%
9.4%
9.5%
9.5%
9.5%
9.5%
9.5%
9.6%
9.6%
9.6%
9.6%
9.6%
9.6%
9.7%
9.7%

9.7%
9.7%
9.7%
9.8%
9.8%
9.8%
9.8%
9.8%
9.9%
9.9%
9.9%
9.9%
9.9%
10.0%
10.0%
10.0%
10.0%
10.0%
10.1%
10.1%
10.1%
10.1%
10.1%
10.1%
10.2%
10.2%
10.2%
10.2%
10.2%
10.3%
10.3%
10.3%
10.3%
10.3%
10.4%
10.4%
10.4%
10.4%
10.4%
10.5%
10.5%
10.5%
10.5%
10.5%
10.6%
10.6%

10.6%
10.6%
10.6%
10.6%
10.7%
10.7%
10.7%
10.7%
10.7%
10.8%
10.8%
10.8%
10.8%
10.8%
10.9%
10.9%
10.9%
10.9%
10.9%
11.0%
11.0%
11.0%
11.0%
11.0%
11.1%
11.1%
11.1%
11.1%
11.1%
11.1%
11.2%
11.2%
11.2%
11.2%
11.2%
11.3%
11.3%
11.3%
11.3%
11.3%
11.4%
11.4%
11.4%
11.4%
11.4%
11.5%

11.5%
11.5%
11.5%
11.5%
11.6%
11.6%
11.6%
11.6%
11.6%
11.6%
11.6%
11.7%
11.7%
11.7%
11.7%
11.7%
11.8%
11.8%
11.8%
11.8%
11.8%
11.8%
11.9%
11.9%
11.9%
11.9%
11.9%
12.0%
12.0%
12.0%
12.0%
12.0%
12.1%
12.1%
12.1%
12.1%
12.1%
12.1%
12.1%
12.2%
12.2%
12.2%
12.2%
12.2%
12.3%
12.3%
12.3%
12.3%
12.3%

12.4%
12.4%
12.4%
12.4%
12.4%
12.5%
12.5%
12.5%
12.5%
12.5%
12.5%
12.6%
12.6%
12.6%
12.6%
12.6%
12.7%
12.7%
12.7%
12.7%
12.7%
12.8%
12.8%
12.8%
12.8%
12.8%
12.9%
12.9%
12.9%
12.9%
12.9%
13.0%
13.0%
13.0%
13.0%
13.0%
13.0%
13.1%
13.1%
13.1%
13.1%
13.1%
13.2%
13.2%
13.2%
13.2%

13.2%
13.3%
13.3%
13.3%
13.3%
13.3%
13.4%
13.4%
13.4%
13.4%
13.4%
13.5%
13.5%
13.5%
13.5%
13.5%
13.5%
13.6%
13.6%
13.6%
13.6%
13.6%
13.7%
13.7%
13.7%
13.7%
13.7%
13.8%
13.8%
13.8%
13.8%
13.8%
13.9%
13.9%
13.9%
13.9%
13.9%
14.0%
14.0%
14.0%
14.0%
14.0%
14.0%
14.1%
14.1%
14.1%

14.1%
14.1%
14.2%
14.2%
14.2%
14.2%
14.2%
14.3%
14.3%
14.3%
14.3%
14.3%
14.4%
14.4%
14.4%
14.4%
14.4%
14.5%
14.5%
14.5%
14.5%
14.5%
14.5%
14.6%
14.6%
14.6%
14.6%
14.6%
14.7%
14.7%
14.7%
14.7%
14.7%
14.8%
14.8%
14.8%
14.8%
14.8%
14.9%
14.9%
14.9%
14.9%
14.9%
15.0%
15.0%
15.0%

15.0%
15.0%
15.0%
15.1%
15.1%
15.1%
15.1%
15.1%
15.2%
15.2%
15.2%
15.2%
15.2%
15.3%
15.3%
15.3%
15.3%
15.3%
15.4%
15.4%
15.4%
15.4%
15.4%
15.5%
15.5%
15.5%
15.5%
15.5%
15.5%
15.6%
15.6%
15.6%
15.6%
15.6%
15.7%
15.7%
15.7%
15.7%
15.7%
15.8%
15.8%
15.8%
15.8%
15.8%
15.9%
15.9%

15.9%
15.9%
15.9%
16.0%
16.0%
16.0%
16.0%
16.0%
16.0%
16.0%
16.1%
16.1%
16.1%
16.1%
16.1%
16.2%
16.2%
16.2%
16.2%
16.2%
16.3%
16.3%
16.3%
16.3%
16.3%
16.4%
16.4%
16.4%
16.4%
16.4%
16.5%
16.5%
16.5%
16.5%
16.5%
16.5%
16.6%
16.6%
16.6%
16.6%
16.6%
16.7%
16.7%
16.7%
16.7%
16.7%
16.8%

16.8%
16.8%
16.8%
16.8%
16.9%
16.9%
16.9%
16.9%
16.9%
17.0%
17.0%
17.0%
17.0%
17.0%
17.0%
17.1%
17.1%
17.1%
17.1%
17.1%
17.2%
17.2%
17.2%
17.2%
17.2%
17.3%
17.3%
17.3%
17.3%
17.3%
17.4%
17.4%
17.4%
17.4%
17.4%
17.5%
17.5%
17.5%
17.5%
17.5%
17.5%
17.6%
17.6%
17.6%
17.6%
17.6%

17.7%
17.7%
17.7%
17.7%
17.7%
17.8%
17.8%
17.8%
17.8%
17.8%
17.9%
17.9%
17.9%
17.9%
17.9%
18.0%
18.0%
18.0%
18.0%
18.0%
18.0%
18.1%
18.1%
18.1%
18.1%
18.1%
18.2%
18.2%
18.2%
18.2%
18.2%
18.3%
18.3%
18.3%
18.3%
18.3%
18.4%
18.4%
18.4%
18.4%
18.4%
18.5%
18.5%
18.5%
18.5%
18.5%

18.5%
18.6%
18.6%
18.6%
18.6%
18.6%
18.7%
18.7%
18.7%
18.7%
18.7%
18.8%
18.8%
18.8%
18.8%
18.8%
18.9%
18.9%
18.9%
18.9%
18.9%
18.9%
19.0%
19.0%
19.0%
19.0%
19.0%
19.1%
19.1%
19.1%
19.1%
19.1%
19.2%
19.2%
19.2%
19.2%
19.2%
19.3%
19.3%
19.3%
19.3%
19.3%
19.4%
19.4%
19.4%
19.4%

19.4%
19.4%
19.5%
19.5%
19.5%
19.5%
19.5%
19.6%
19.6%
19.6%
19.6%
19.6%
19.7%
19.7%
19.7%
19.7%
19.7%
19.8%
19.8%
19.8%
19.8%
19.8%
19.9%
19.9%
19.9%
19.9%
19.9%
19.9%
20.0%
20.0%
20.0%
20.0%
20.0%
20.1%
20.1%
20.1%
20.1%
20.1%
20.2%
20.2%
20.2%
20.2%
20.2%
20.3%
20.3%
20.3%

20.3%
20.3%
20.4%
20.4%
20.4%
20.4%
20.4%
20.4%
20.5%
20.5%
20.5%
20.5%
20.5%
20.6%
20.6%
20.6%
20.6%
20.6%
20.7%
20.7%
20.7%
20.7%
20.7%
20.8%
20.8%
20.8%
20.8%
20.8%
20.9%
20.9%
20.9%
20.9%
20.9%
20.9%
21.0%
21.0%
21.0%
21.0%
21.0%
21.1%
21.1%
21.1%
21.1%
21.1%
21.2%
21.2%

21.2%
21.2%
21.2%
21.3%
21.3%
21.3%
21.3%
21.3%
21.4%
21.4%
21.4%
21.4%
21.4%
21.4%
21.5%
21.5%
21.5%
21.5%
21.5%
21.6%
21.6%
21.6%
21.6%
21.6%
21.7%
21.7%
21.7%
21.7%
21.7%
21.8%
21.8%
21.8%
21.8%
21.8%
21.9%
21.9%
21.9%
21.9%
21.9%
21.9%
22.0%
22.0%
22.0%
22.0%
22.0%
22.1%

22.1%
22.1%
22.1%
22.1%
22.2%
22.2%
22.2%
22.2%
22.2%
22.3%
22.3%
22.3%
22.3%
22.3%
22.4%
22.4%
22.4%
22.4%
22.4%
22.4%
22.4%
22.5%
22.5%
22.5%
22.5%
22.5%
22.6%
22.6%
22.6%
22.6%
22.6%
22.7%
22.7%
22.7%
22.7%
22.7%
22.8%
22.8%
22.8%
22.8%
22.8%
22.9%
22.9%
22.9%
22.9%
22.9%
22.9%

23.0%
23.0%
23.0%
23.0%
23.0%
23.1%
23.1%
23.1%
23.1%
23.1%
23.2%
23.2%
23.2%
23.2%
23.2%
23.3%
23.3%
23.3%
23.3%
23.3%
23.4%
23.4%
23.4%
23.4%
23.4%
23.4%
23.5%
23.5%
23.5%
23.5%
23.5%
23.6%
23.6%
23.6%
23.6%
23.6%
23.7%
23.7%
23.7%
23.7%
23.7%
23.8%
23.8%
23.8%
23.8%
23.8%

23.9%
23.9%
23.9%
23.9%
23.9%
23.9%
24.0%
24.0%
24.0%
24.0%
24.0%
24.1%
24.1%
24.1%
24.1%
24.1%
24.2%
24.2%
24.2%
24.2%
24.2%
24.3%
24.3%
24.3%
24.3%
24.3%
24.4%
24.4%
24.4%
24.4%
24.4%
24.4%
24.4%
24.5%
24.5%
24.5%
24.5%
24.5%
24.6%
24.6%
24.6%
24.6%
24.6%
24.7%
24.7%
24.7%
24.7%

24.7%
24.8%
24.8%
24.8%
24.8%
24.8%
24.9%
24.9%
24.9%
24.9%
24.9%
24.9%
25.0%
25.0%
25.0%
25.0%
25.0%
25.1%
25.1%
25.1%
25.1%
25.1%
25.2%
25.2%
25.2%
25.2%
25.2%
25.3%
25.3%
25.3%
25.3%
25.3%
25.3%
25.4%
25.4%
25.4%
25.4%
25.4%
25.5%
25.5%
25.5%
25.5%
25.5%
25.6%
25.6%
25.6%

25.6%
25.6%
25.7%
25.7%
25.7%
25.7%
25.7%
25.8%
25.8%
25.8%
25.8%
25.8%
25.8%
25.9%
25.9%
25.9%
25.9%
25.9%
26.0%
26.0%
26.0%
26.0%
26.0%
26.1%
26.1%
26.1%
26.1%
26.1%
26.2%
26.2%
26.2%
26.2%
26.2%
26.3%
26.3%
26.3%
26.3%
26.3%
26.3%
26.4%
26.4%
26.4%
26.4%
26.4%
26.5%
26.5%

26.5%
26.5%
26.5%
26.6%
26.6%
26.6%
26.6%
26.6%
26.6%
26.7%
26.7%
26.7%
26.7%
26.7%
26.7%
26.8%
26.8%
26.8%
26.8%
26.8%
26.8%
26.8%
26.9%
26.9%
26.9%
26.9%
26.9%
27.0%
27.0%
27.0%
27.0%
27.0%
27.0%
27.1%
27.1%
27.1%
27.1%
27.1%
27.1%
27.2%
27.2%
27.2%
27.2%
27.2%
27.2%
27.3%
27.3%
27.3%
27.3%
27.3%
27.3%
27.3%
27.4%

27.4%
27.4%
27.4%
27.4%
27.5%
27.5%
27.5%
27.5%
27.5%
27.6%
27.6%
27.6%
27.6%
27.6%
27.7%
27.7%
27.7%
27.7%
27.7%
27.8%
27.8%
27.8%
27.8%
27.8%
27.8%
27.8%
27.9%
27.9%
27.9%
27.9%
27.9%
28.0%
28.0%
28.0%
28.0%
28.0%
28.1%
28.1%
28.1%
28.1%
28.1%
28.2%
28.2%
28.2%
28.2%
28.2%
28.3%

28.3%
28.3%
28.3%
28.3%
28.3%
28.4%
28.4%
28.4%
28.4%
28.4%
28.5%
28.5%
28.5%
28.5%
28.5%
28.6%
28.6%
28.6%
28.6%
28.6%
28.7%
28.7%
28.7%
28.7%
28.7%
28.8%
28.8%
28.8%
28.8%
28.8%
28.8%
28.8%
28.9%
28.9%
28.9%
28.9%
28.9%
29.0%
29.0%
29.0%
29.0%
29.0%
29.1%
29.1%
29.1%
29.1%
29.1%

29.2%
29.2%
29.2%
29.2%
29.2%
29.3%
29.3%
29.3%
29.3%
29.3%
29.3%
29.4%
29.4%
29.4%
29.4%
29.4%
29.5%
29.5%
29.5%
29.5%
29.5%
29.6%
29.6%
29.6%
29.6%
29.6%
29.7%
29.7%
29.7%
29.7%
29.7%
29.8%
29.8%
29.8%
29.8%
29.8%
29.8%
29.9%
29.9%
29.9%
29.9%
29.9%
30.0%
30.0%
30.0%
30.0%

30.0%
30.1%
30.1%
30.1%
30.1%
30.1%
30.2%
30.2%
30.2%
30.2%
30.2%
30.3%
30.3%
30.3%
30.3%
30.3%
30.3%
30.4%
30.4%
30.4%
30.4%
30.4%
30.5%
30.5%
30.5%
30.5%
30.5%
30.6%
30.6%
30.6%
30.6%
30.6%
30.7%
30.7%
30.7%
30.7%
30.7%
30.8%
30.8%
30.8%
30.8%
30.8%
30.8%
30.9%
30.9%
30.9%

30.9%
30.9%
31.0%
31.0%
31.0%
31.0%
31.0%
31.1%
31.1%
31.1%
31.1%
31.1%
31.2%
31.2%
31.2%
31.2%
31.2%
31.3%
31.3%
31.3%
31.3%
31.3%
31.3%
31.4%
31.4%
31.4%
31.4%
31.4%
31.5%
31.5%
31.5%
31.5%
31.5%
31.6%
31.6%
31.6%
31.6%
31.6%
31.7%
31.7%
31.7%
31.7%
31.7%
31.7%
31.8%
31.8%

31.8%
31.8%
31.8%
31.9%
31.9%
31.9%
31.9%
31.9%
32.0%
32.0%
32.0%
32.0%
32.0%
32.1%
32.1%
32.1%
32.1%
32.1%
32.2%
32.2%
32.2%
32.2%
32.2%
32.2%
32.3%
32.3%
32.3%
32.3%
32.3%
32.4%
32.4%
32.4%
32.4%
32.4%
32.5%
32.5%
32.5%
32.5%
32.5%
32.6%
32.6%
32.6%
32.6%
32.6%
32.7%
32.7%

32.7%
32.7%
32.7%
32.7%
32.8%
32.8%
32.8%
32.8%
32.8%
32.9%
32.9%
32.9%
32.9%
32.9%
33.0%
33.0%
33.0%
33.0%
33.0%
33.1%
33.1%
33.1%
33.1%
33.1%
33.2%
33.2%
33.2%
33.2%
33.2%
33.2%
33.3%
33.3%
33.3%
33.3%
33.3%
33.4%
33.4%
33.4%
33.4%
33.4%
33.5%
33.5%
33.5%
33.5%
33.5%
33.6%

33.6%
33.6%
33.6%
33.6%
33.7%
33.7%
33.7%
33.7%
33.7%
33.7%
33.8%
33.8%
33.8%
33.8%
33.8%
33.9%
33.9%
33.9%
33.9%
33.9%
34.0%
34.0%
34.0%
34.0%
34.0%
34.1%
34.1%
34.1%
34.1%
34.1%
34.2%
34.2%
34.2%
34.2%
34.2%
34.2%
34.3%
34.3%
34.3%
34.3%
34.3%
34.4%
34.4%
34.4%
34.4%
34.4%

34.5%
34.5%
34.5%
34.5%
34.5%
34.6%
34.6%
34.6%
34.6%
34.6%
34.7%
34.7%
34.7%
34.7%
34.7%
34.7%
34.8%
34.8%
34.8%
34.8%
34.8%
34.9%
34.9%
34.9%
34.9%
34.9%
35.0%
35.0%
35.0%
35.0%
35.0%
35.1%
35.1%
35.1%
35.1%
35.1%
35.2%
35.2%
35.2%
35.2%
35.2%
35.2%
35.3%
35.3%
35.3%
35.3%

35.3%
35.4%
35.4%
35.4%
35.4%
35.4%
35.5%
35.5%
35.5%
35.5%
35.5%
35.6%
35.6%
35.6%
35.6%
35.6%
35.7%
35.7%
35.7%
35.7%
35.7%
35.7%
35.8%
35.8%
35.8%
35.8%
35.8%
35.9%
35.9%
35.9%
35.9%
35.9%
36.0%
36.0%
36.0%
36.0%
36.0%
36.1%
36.1%
36.1%
36.1%
36.1%
36.2%
36.2%
36.2%
36.2%

36.2%
36.2%
36.3%
36.3%
36.3%
36.3%
36.3%
36.4%
36.4%
36.4%
36.4%
36.4%
36.5%
36.5%
36.5%
36.5%
36.5%
36.6%
36.6%
36.6%
36.6%
36.6%
36.7%
36.7%
36.7%
36.7%
36.7%
36.7%
36.8%
36.8%
36.8%
36.8%
36.8%
36.9%
36.9%
36.9%
36.9%
36.9%
37.0%
37.0%
37.0%
37.0%
37.0%
37.1%
37.1%
37.1%

37.1%
37.1%
37.2%
37.2%
37.2%
37.2%
37.2%
37.2%
37.3%
37.3%
37.3%
37.3%
37.3%
37.4%
37.4%
37.4%
37.4%
37.4%
37.5%
37.5%
37.5%
37.5%
37.5%
37.6%
37.6%
37.6%
37.6%
37.6%
37.6%
37.7%
37.7%
37.7%
37.7%
37.7%
37.8%
37.8%
37.8%
37.8%
37.8%
37.9%
37.9%
37.9%
37.9%
37.9%
38.0%
38.0%

38.0%
38.0%
38.0%
38.1%
38.1%
38.1%
38.1%
38.1%
38.1%
38.2%
38.2%
38.2%
38.2%
38.2%
38.3%
38.3%
38.3%
38.3%
38.3%
38.4%
38.4%
38.4%
38.4%
38.4%
38.5%
38.5%
38.5%
38.5%
38.5%
38.6%
38.6%
38.6%
38.6%
38.6%
38.6%
38.7%
38.7%
38.7%
38.7%
38.7%
38.8%
38.8%
38.8%
38.8%
38.8%
38.9%

38.9%
38.9%
38.9%
38.9%
39.0%
39.0%
39.0%
39.0%
39.0%
39.1%
39.1%
39.1%
39.1%
39.1%
39.1%
39.2%
39.2%
39.2%
39.2%
39.2%
39.3%
39.3%
39.3%
39.3%
39.3%
39.4%
39.4%
39.4%
39.4%
39.4%
39.5%
39.5%
39.5%
39.5%
39.5%
39.6%
39.6%
39.6%
39.6%
39.6%
39.6%
39.7%
39.7%
39.7%
39.7%
39.7%

39.8%
39.8%
39.8%
39.8%
39.8%
39.9%
39.9%
39.9%
39.9%
39.9%
40.0%
40.0%
40.0%
40.0%
40.0%
40.1%
40.1%
40.1%
40.1%
40.1%
40.1%
40.2%
40.2%
40.2%
40.2%
40.2%
40.3%
40.3%
40.3%
40.3%
40.3%
40.4%
40.4%
40.4%
40.4%
40.4%
40.5%
40.5%
40.5%
40.5%
40.5%
40.6%
40.6%
40.6%
40.6%
40.6%

40.6%
40.7%
40.7%
40.7%
40.7%
40.7%
40.8%
40.8%
40.8%
40.8%
40.8%
40.9%
40.9%
40.9%
40.9%
40.9%
41.0%
41.0%
41.0%
41.0%
41.0%
41.1%
41.1%
41.1%
41.1%
41.1%
41.1%
41.2%
41.2%
41.2%
41.2%
41.2%
41.3%
41.3%
41.3%
41.3%
41.3%
41.3%
41.4%
41.4%
41.4%
41.4%
41.4%
41.5%
41.5%
41.5%
41.5%

41.5%
41.6%
41.6%
41.6%
41.6%
41.6%
41.6%
41.7%
41.7%
41.7%
41.7%
41.7%
41.8%
41.8%
41.8%
41.8%
41.8%
41.9%
41.9%
41.9%
41.9%
41.9%
42.0%
42.0%
42.0%
42.0%
42.0%
42.1%
42.1%
42.1%
42.1%
42.1%
42.1%
42.2%
42.2%
42.2%
42.2%
42.2%
42.3%
42.3%
42.3%
42.3%
42.3%
42.4%
42.4%
42.4%

42.4%
42.4%
42.5%
42.5%
42.5%
42.5%
42.5%
42.6%
42.6%
42.6%
42.6%
42.6%
42.6%
42.7%
42.7%
42.7%
42.7%
42.7%
42.8%
42.8%
42.8%
42.8%
42.8%
42.9%
42.9%
42.9%
42.9%
42.9%
43.0%
43.0%
43.0%
43.0%
43.0%
43.1%
43.1%
43.1%
43.1%
43.1%
43.1%
43.2%
43.2%
43.2%
43.2%
43.2%
43.3%
43.3%

43.3%
43.3%
43.3%
43.4%
43.4%
43.4%
43.4%
43.4%
43.5%
43.5%
43.5%
43.5%
43.5%
43.6%
43.6%
43.6%
43.6%
43.6%
43.6%
43.6%
43.7%
43.7%
43.7%
43.7%
43.7%
43.8%
43.8%
43.8%
43.8%
43.8%
43.9%
43.9%
43.9%
43.9%
43.9%
44.0%
44.0%
44.0%
44.0%
44.0%
44.0%
44.1%
44.1%
44.1%
44.1%
44.1%
44.2%

44.2%
44.2%
44.2%
44.2%
44.3%
44.3%
44.3%
44.3%
44.3%
44.4%
44.4%
44.4%
44.4%
44.4%
44.5%
44.5%
44.5%
44.5%
44.5%
44.5%
44.5%
44.6%
44.6%
44.6%
44.6%
44.6%
44.6%
44.7%
44.7%
44.7%
44.7%
44.7%
44.8%
44.8%
44.8%
44.8%
44.8%
44.8%
44.9%
44.9%
44.9%
44.9%
44.9%
45.0%
45.0%
45.0%
45.0%
45.0%
45.0%

45.1%
45.1%
45.1%
45.1%
45.1%
45.2%
45.2%
45.2%
45.2%
45.2%
45.3%
45.3%
45.3%
45.3%
45.3%
45.4%
45.4%
45.4%
45.4%
45.4%
45.5%
45.5%
45.5%
45.5%
45.5%
45.5%
45.6%
45.6%
45.6%
45.6%
45.6%
45.7%
45.7%
45.7%
45.7%
45.7%
45.8%
45.8%
45.8%
45.8%
45.8%
45.9%
45.9%
45.9%
45.9%
45.9%

46.0%
46.0%
46.0%
46.0%
46.0%
46.0%
46.1%
46.1%
46.1%
46.1%
46.1%
46.2%
46.2%
46.2%
46.2%
46.2%
46.3%
46.3%
46.3%
46.3%
46.3%
46.4%
46.4%
46.4%
46.4%
46.4%
46.5%
46.5%
46.5%
46.5%
46.5%
46.5%
46.6%
46.6%
46.6%
46.6%
46.6%
46.7%
46.7%
46.7%
46.7%
46.7%
46.8%
46.8%
46.8%
46.8%

46.8%
46.9%
46.9%
46.9%
46.9%
46.9%
47.0%
47.0%
47.0%
47.0%
47.0%
47.0%
47.1%
47.1%
47.1%
47.1%
47.1%
47.2%
47.2%
47.2%
47.2%
47.2%
47.3%
47.3%
47.3%
47.3%
47.3%
47.4%
47.4%
47.4%
47.4%
47.4%
47.5%
47.5%
47.5%
47.5%
47.5%
47.5%
47.6%
47.6%
47.6%
47.6%
47.6%
47.7%
47.7%
47.7%

47.7%
47.7%
47.8%
47.8%
47.8%
47.8%
47.8%
47.9%
47.9%
47.9%
47.9%
47.9%
48.0%
48.0%
48.0%
48.0%
48.0%
48.0%
48.1%
48.1%
48.1%
48.1%
48.1%
48.2%
48.2%
48.2%
48.2%
48.2%
48.3%
48.3%
48.3%
48.3%
48.3%
48.4%
48.4%
48.4%
48.4%
48.4%
48.5%
48.5%
48.5%
48.5%
48.5%
48.5%
48.6%
48.6%

48.6%
48.6%
48.6%
48.7%
48.7%
48.7%
48.7%
48.7%
48.8%
48.8%
48.8%
48.8%
48.8%
48.9%
48.9%
48.9%
48.9%
48.9%
49.0%
49.0%
49.0%
49.0%
49.0%
49.0%
49.1%
49.1%
49.1%
49.1%
49.1%
49.2%
49.2%
49.2%
49.2%
49.2%
49.3%
49.3%
49.3%
49.3%
49.3%
49.4%
49.4%
49.4%
49.4%
49.4%
49.5%
49.5%

49.5%
49.5%
49.5%
49.5%
49.6%
49.6%
49.6%
49.6%
49.6%
49.6%
49.7%
49.7%
49.7%
49.7%
49.7%
49.8%
49.8%
49.8%
49.8%
49.8%
49.9%
49.9%
49.9%
49.9%
49.9%
50.0%
50.0%
50.0%
50.0%
50.0%
50.0%
50.1%
50.1%
50.1%
50.1%
50.1%
50.2%
50.2%
50.2%
50.2%
50.2%
50.3%
50.3%
50.3%
50.3%
50.3%
50.4%

50.4%
50.4%
50.4%
50.4%
50.4%
50.5%
50.5%
50.5%
50.5%
50.5%
50.6%
50.6%
50.6%
50.6%
50.6%
50.7%
50.7%
50.7%
50.7%
50.7%
50.8%
50.8%
50.8%
50.8%
50.8%
50.9%
50.9%
50.9%
50.9%
50.9%
50.9%
51.0%
51.0%
51.0%
51.0%
51.0%
51.1%
51.1%
51.1%
51.1%
51.1%
51.2%
51.2%
51.2%
51.2%
51.2%

51.3%
51.3%
51.3%
51.3%
51.3%
51.4%
51.4%
51.4%
51.4%
51.4%
51.4%
51.5%
51.5%
51.5%
51.5%
51.5%
51.6%
51.6%
51.6%
51.6%
51.6%
51.7%
51.7%
51.7%
51.7%
51.7%
51.8%
51.8%
51.8%
51.8%
51.8%
51.9%
51.9%
51.9%
51.9%
51.9%
51.9%
52.0%
52.0%
52.0%
52.0%
52.0%
52.1%
52.1%
52.1%
52.1%

52.1%
52.2%
52.2%
52.2%
52.2%
52.2%
52.3%
52.3%
52.3%
52.3%
52.3%
52.4%
52.4%
52.4%
52.4%
52.4%
52.4%
52.5%
52.5%
52.5%
52.5%
52.5%
52.6%
52.6%
52.6%
52.6%
52.6%
52.7%
52.7%
52.7%
52.7%
52.7%
52.8%
52.8%
52.8%
52.8%
52.8%
52.8%
52.9%
52.9%
52.9%
52.9%
52.9%
52.9%
53.0%
53.0%
53.0%

53.0%
53.0%
53.1%
53.1%
53.1%
53.1%
53.1%
53.2%
53.2%
53.2%
53.2%
53.2%
53.3%
53.3%
53.3%
53.3%
53.3%
53.4%
53.4%
53.4%
53.4%
53.4%
53.4%
53.5%
53.5%
53.5%
53.5%
53.5%
53.6%
53.6%
53.6%
53.6%
53.6%
53.7%
53.7%
53.7%
53.7%
53.7%
53.8%
53.8%
53.8%
53.8%
53.8%
53.9%
53.9%
53.9%

53.9%
53.9%
53.9%
54.0%
54.0%
54.0%
54.0%
54.0%
54.1%
54.1%
54.1%
54.1%
54.1%
54.2%
54.2%
54.2%
54.2%
54.2%
54.3%
54.3%
54.3%
54.3%
54.3%
54.4%
54.4%
54.4%
54.4%
54.4%
54.4%
54.5%
54.5%
54.5%
54.5%
54.5%
54.6%
54.6%
54.6%
54.6%
54.6%
54.7%
54.7%
54.7%
54.7%
54.7%
54.8%
54.8%

54.8%
54.8%
54.8%
54.9%
54.9%
54.9%
54.9%
54.9%
54.9%
54.9%
55.0%
55.0%
55.0%
55.0%
55.0%
55.1%
55.1%
55.1%
55.1%
55.1%
55.2%
55.2%
55.2%
55.2%
55.2%
55.3%
55.3%
55.3%
55.3%
55.3%
55.4%
55.4%
55.4%
55.4%
55.4%
55.4%
55.5%
55.5%
55.5%
55.5%
55.5%
55.6%
55.6%
55.6%
55.6%
55.6%
55.7%

55.7%
55.7%
55.7%
55.7%
55.8%
55.8%
55.8%
55.8%
55.8%
55.9%
55.9%
55.9%
55.9%
55.9%
55.9%
56.0%
56.0%
56.0%
56.0%
56.0%
56.1%
56.1%
56.1%
56.1%
56.1%
56.2%
56.2%
56.2%
56.2%
56.2%
56.3%
56.3%
56.3%
56.3%
56.3%
56.4%
56.4%
56.4%
56.4%
56.4%
56.4%
56.5%
56.5%
56.5%
56.5%
56.5%

56.6%
56.6%
56.6%
56.6%
56.6%
56.7%
56.7%
56.7%
56.7%
56.7%
56.8%
56.8%
56.8%
56.8%
56.8%
56.8%
56.9%
56.9%
56.9%
56.9%
56.9%
57.0%
57.0%
57.0%
57.0%
57.0%
57.1%
57.1%
57.1%
57.1%
57.1%
57.2%
57.2%
57.2%
57.2%
57.2%
57.3%
57.3%
57.3%
57.3%
57.3%
57.3%
57.4%
57.4%
57.4%
57.4%

57.4%
57.5%
57.5%
57.5%
57.5%
57.5%
57.6%
57.6%
57.6%
57.6%
57.6%
57.7%
57.7%
57.7%
57.7%
57.7%
57.8%
57.8%
57.8%
57.8%
57.8%
57.8%
57.8%
57.9%
57.9%
57.9%
57.9%
57.9%
58.0%
58.0%
58.0%
58.0%
58.0%
58.1%
58.1%
58.1%
58.1%
58.1%
58.2%
58.2%
58.2%
58.2%
58.2%
58.3%
58.3%
58.3%
58.3%

58.3%
58.3%
58.4%
58.4%
58.4%
58.4%
58.4%
58.5%
58.5%
58.5%
58.5%
58.5%
58.6%
58.6%
58.6%
58.6%
58.6%
58.7%
58.7%
58.7%
58.7%
58.7%
58.8%
58.8%
58.8%
58.8%
58.8%
58.8%
58.9%
58.9%
58.9%
58.9%
58.9%
59.0%
59.0%
59.0%
59.0%
59.0%
59.1%
59.1%
59.1%
59.1%
59.1%
59.2%
59.2%
59.2%

59.2%
59.2%
59.3%
59.3%
59.3%
59.3%
59.3%
59.3%
59.4%
59.4%
59.4%
59.4%
59.4%
59.5%
59.5%
59.5%
59.5%
59.5%
59.6%
59.6%
59.6%
59.6%
59.6%
59.7%
59.7%
59.7%
59.7%
59.7%
59.8%
59.8%
59.8%
59.8%
59.8%
59.8%
59.9%
59.9%
59.9%
59.9%
59.9%
60.0%
60.0%
60.0%
60.0%
60.0%
60.1%
60.1%

60.1%
60.1%
60.1%
60.2%
60.2%
60.2%
60.2%
60.2%
60.3%
60.3%
60.3%
60.3%
60.3%
60.3%
60.4%
60.4%
60.4%
60.4%
60.4%
60.5%
60.5%
60.5%
60.5%
60.5%
60.6%
60.6%
60.6%
60.6%
60.6%
60.7%
60.7%
60.7%
60.7%
60.7%
60.8%
60.8%
60.8%
60.8%
60.8%
60.8%
60.9%
60.9%
60.9%
60.9%
60.9%
61.0%

61.0%
61.0%
61.0%
61.0%
61.1%
61.1%
61.1%
61.1%
61.1%
61.2%
61.2%
61.2%
61.2%
61.2%
61.3%
61.3%
61.3%
61.3%
61.3%
61.3%
61.3%
61.4%
61.4%
61.4%
61.4%
61.4%
61.5%
61.5%
61.5%
61.5%
61.5%
61.6%
61.6%
61.6%
61.6%
61.6%
61.7%
61.7%
61.7%
61.7%
61.7%
61.8%
61.8%
61.8%
61.8%
61.8%
61.8%

61.9%
61.9%
61.9%
61.9%
61.9%
62.0%
62.0%
62.0%
62.0%
62.0%
62.1%
62.1%
62.1%
62.1%
62.1%
62.2%
62.2%
62.2%
62.2%
62.2%
62.3%
62.3%
62.3%
62.3%
62.3%
62.3%
62.4%
62.4%
62.4%
62.4%
62.4%
62.5%
62.5%
62.5%
62.5%
62.5%
62.6%
62.6%
62.6%
62.6%
62.6%
62.7%
62.7%
62.7%
62.7%
62.7%

62.7%
62.8%
62.8%
62.8%
62.8%
62.8%
62.9%
62.9%
62.9%
62.9%
62.9%
63.0%
63.0%
63.0%
63.0%
63.0%
63.1%
63.1%
63.1%
63.1%
63.1%
63.2%
63.2%
63.2%
63.2%
63.2%
63.2%
63.3%
63.3%
63.3%
63.3%
63.3%
63.4%
63.4%
63.4%
63.4%
63.4%
63.5%
63.5%
63.5%
63.5%
63.5%
63.6%
63.6%
63.6%
63.6%

63.6%
63.7%
63.7%
63.7%
63.7%
63.7%
63.7%
63.8%
63.8%
63.8%
63.8%
63.8%
63.9%
63.9%
63.9%
63.9%
63.9%
64.0%
64.0%
64.0%
64.0%
64.0%
64.1%
64.1%
64.1%
64.1%
64.1%
64.2%
64.2%
64.2%
64.2%
64.2%
64.2%
64.3%
64.3%
64.3%
64.3%
64.3%
64.4%
64.4%
64.4%
64.4%
64.4%
64.5%
64.5%
64.5%

64.5%
64.5%
64.6%
64.6%
64.6%
64.6%
64.6%
64.7%
64.7%
64.7%
64.7%
64.7%
64.7%
64.8%
64.8%
64.8%
64.8%
64.8%
64.9%
64.9%
64.9%
64.9%
64.9%
65.0%
65.0%
65.0%
65.0%
65.0%
65.1%
65.1%
65.1%
65.1%
65.1%
65.2%
65.2%
65.2%
65.2%
65.2%
65.2%
65.3%
65.3%
65.3%
65.3%
65.3%
65.4%
65.4%

65.4%
65.4%
65.4%
65.5%
65.5%
65.5%
65.5%
65.5%
65.6%
65.6%
65.6%
65.6%
65.6%
65.7%
65.7%
65.7%
65.7%
65.7%
65.7%
65.8%
65.8%
65.8%
65.8%
65.8%
65.9%
65.9%
65.9%
65.9%
65.9%
66.0%
66.0%
66.0%
66.0%
66.0%
66.1%
66.1%
66.1%
66.1%
66.1%
66.2%
66.2%
66.2%
66.2%
66.2%
66.2%
66.3%

66.3%
66.3%
66.3%
66.3%
66.4%
66.4%
66.4%
66.4%
66.4%
66.5%
66.5%
66.5%
66.5%
66.5%
66.6%
66.6%
66.6%
66.6%
66.6%
66.7%
66.7%
66.7%
66.7%
66.7%
66.7%
66.8%
66.8%
66.8%
66.8%
66.8%
66.9%
66.9%
66.9%
66.9%
66.9%
67.0%
67.0%
67.0%
67.0%
67.0%
67.1%
67.1%
67.1%
67.1%
67.1%
67.2%

67.2%
67.2%
67.2%
67.2%
67.2%
67.3%
67.3%
67.3%
67.3%
67.3%
67.4%
67.4%
67.4%
67.4%
67.4%
67.5%
67.5%
67.5%
67.5%
67.5%
67.6%
67.6%
67.6%
67.6%
67.6%
67.7%
67.7%
67.7%
67.7%
67.7%
67.7%
67.8%
67.8%
67.8%
67.8%
67.8%
67.9%
67.9%
67.9%
67.9%
67.9%
68.0%
68.0%
68.0%
68.0%
68.0%

68.1%
68.1%
68.1%
68.1%
68.1%
68.2%
68.2%
68.2%
68.2%
68.2%
68.2%
68.3%
68.3%
68.3%
68.3%
68.3%
68.4%
68.4%
68.4%
68.4%
68.4%
68.5%
68.5%
68.5%
68.5%
68.5%
68.6%
68.6%
68.6%
68.6%
68.6%
68.7%
68.7%
68.7%
68.7%
68.7%
68.7%
68.8%
68.8%
68.8%
68.8%
68.8%
68.9%
68.9%
68.9%
68.9%

68.9%
69.0%
69.0%
69.0%
69.0%
69.0%
69.1%
69.1%
69.1%
69.1%
69.1%
69.1%
69.2%
69.2%
69.2%
69.2%
69.2%
69.3%
69.3%
69.3%
69.3%
69.3%
69.4%
69.4%
69.4%
69.4%
69.4%
69.5%
69.5%
69.5%
69.5%
69.5%
69.6%
69.6%
69.6%
69.6%
69.6%
69.6%
69.6%
69.7%
69.7%
69.7%
69.7%
69.7%
69.8%
69.8%
69.8%

69.8%
69.8%
69.9%
69.9%
69.9%
69.9%
69.9%
70.0%
70.0%
70.0%
70.0%
70.0%
70.1%
70.1%
70.1%
70.1%
70.1%
70.1%
70.2%
70.2%
70.2%
70.2%
70.2%
70.3%
70.3%
70.3%
70.3%
70.3%
70.4%
70.4%
70.4%
70.4%
70.4%
70.5%
70.5%
70.5%
70.5%
70.5%
70.6%
70.6%
70.6%
70.6%
70.6%
70.6%
70.7%
70.7%

70.7%
70.7%
70.7%
70.8%
70.8%
70.8%
70.8%
70.8%
70.8%
70.9%
70.9%
70.9%
70.9%
70.9%
71.0%
71.0%
71.0%
71.0%
71.0%
71.1%
71.1%
71.1%
71.1%
71.1%
71.1%
71.2%
71.2%
71.2%
71.2%
71.2%
71.3%
71.3%
71.3%
71.3%
71.3%
71.4%
71.4%
71.4%
71.4%
71.4%
71.5%
71.5%
71.5%
71.5%
71.5%
71.6%
71.6%

71.6%
71.6%
71.6%
71.6%
71.7%
71.7%
71.7%
71.7%
71.7%
71.8%
71.8%
71.8%
71.8%
71.8%
71.9%
71.9%
71.9%
71.9%
71.9%
72.0%
72.0%
72.0%
72.0%
72.0%
72.1%
72.1%
72.1%
72.1%
72.1%
72.1%
72.2%
72.2%
72.2%
72.2%
72.2%
72.3%
72.3%
72.3%
72.3%
72.3%
72.4%
72.4%
72.4%
72.4%
72.4%
72.5%

72.5%
72.5%
72.5%
72.5%
72.6%
72.6%
72.6%
72.6%
72.6%
72.6%
72.7%
72.7%
72.7%
72.7%
72.7%
72.8%
72.8%
72.8%
72.8%
72.8%
72.9%
72.9%
72.9%
72.9%
72.9%
73.0%
73.0%
73.0%
73.0%
73.0%
73.1%
73.1%
73.1%
73.1%
73.1%
73.1%
73.2%
73.2%
73.2%
73.2%
73.2%
73.3%
73.3%
73.3%
73.3%
73.3%

73.4%
73.4%
73.4%
73.4%
73.4%
73.5%
73.5%
73.5%
73.5%
73.5%
73.6%
73.6%
73.6%
73.6%
73.6%
73.6%
73.6%
73.7%
73.7%
73.7%
73.7%
73.7%
73.8%
73.8%
73.8%
73.8%
73.8%
73.8%
73.9%
73.9%
73.9%
73.9%
73.9%
74.0%
74.0%
74.0%
74.0%
74.0%
74.1%
74.1%
74.1%
74.1%
74.1%
74.1%
74.2%
74.2%
74.2%
74.2%

74.2%
74.3%
74.3%
74.3%
74.3%
74.3%
74.4%
74.4%
74.4%
74.4%
74.4%
74.5%
74.5%
74.5%
74.5%
74.5%
74.6%
74.6%
74.6%
74.6%
74.6%
74.6%
74.7%
74.7%
74.7%
74.7%
74.7%
74.8%
74.8%
74.8%
74.8%
74.8%
74.9%
74.9%
74.9%
74.9%
74.9%
75.0%
75.0%
75.0%
75.0%
75.0%
75.1%
75.1%
75.1%
75.1%

75.1%
75.1%
75.2%
75.2%
75.2%
75.2%
75.2%
75.3%
75.3%
75.3%
75.3%
75.3%
75.4%
75.4%
75.4%
75.4%
75.4%
75.5%
75.5%
75.5%
75.5%
75.5%
75.5%
75.6%
75.6%
75.6%
75.6%
75.6%
75.7%
75.7%
75.7%
75.7%
75.7%
75.8%
75.8%
75.8%
75.8%
75.8%
75.9%
75.9%
75.9%
75.9%
75.9%
76.0%
76.0%
76.0%

76.0%
76.0%
76.0%
76.1%
76.1%
76.1%
76.1%
76.1%
76.2%
76.2%
76.2%
76.2%
76.2%
76.3%
76.3%
76.3%
76.3%
76.3%
76.4%
76.4%
76.4%
76.4%
76.4%
76.5%
76.5%
76.5%
76.5%
76.5%
76.5%
76.6%
76.6%
76.6%
76.6%
76.6%
76.7%
76.7%
76.7%
76.7%
76.7%
76.8%
76.8%
76.8%
76.8%
76.8%
76.9%
76.9%

76.9%
76.9%
76.9%
77.0%
77.0%
77.0%
77.0%
77.0%
77.0%
77.0%
77.1%
77.1%
77.1%
77.1%
77.1%
77.2%
77.2%
77.2%
77.2%
77.2%
77.3%
77.3%
77.3%
77.3%
77.3%
77.4%
77.4%
77.4%
77.4%
77.4%
77.5%
77.5%
77.5%
77.5%
77.5%
77.5%
77.6%
77.6%
77.6%
77.6%
77.6%
77.7%
77.7%
77.7%
77.7%
77.7%
77.8%

77.8%
77.8%
77.8%
77.8%
77.9%
77.9%
77.9%
77.9%
77.9%
78.0%
78.0%
78.0%
78.0%
78.0%
78.0%
78.1%
78.1%
78.1%
78.1%
78.1%
78.2%
78.2%
78.2%
78.2%
78.2%
78.3%
78.3%
78.3%
78.3%
78.3%
78.4%
78.4%
78.4%
78.4%
78.4%
78.5%
78.5%
78.5%
78.5%
78.5%
78.5%
78.6%
78.6%
78.6%
78.6%
78.6%

78.7%
78.7%
78.7%
78.7%
78.7%
78.8%
78.8%
78.8%
78.8%
78.8%
78.9%
78.9%
78.9%
78.9%
78.9%
79.0%
79.0%
79.0%
79.0%
79.0%
79.0%
79.1%
79.1%
79.1%
79.1%
79.1%
79.2%
79.2%
79.2%
79.2%
79.2%
79.3%
79.3%
79.3%
79.3%
79.3%
79.4%
79.4%
79.4%
79.4%
79.4%
79.5%
79.5%
79.5%
79.5%
79.5%

79.5%
79.6%
79.6%
79.6%
79.6%
79.6%
79.7%
79.7%
79.7%
79.7%
79.7%
79.8%
79.8%
79.8%
79.8%
79.8%
79.9%
79.9%
79.9%
79.9%
79.9%
80.0%
80.0%
80.0%
80.0%
80.0%
80.0%
80.1%
80.1%
80.1%
80.1%
80.1%
80.2%
80.2%
80.2%
80.2%
80.2%
80.3%
80.3%
80.3%
80.3%
80.3%
80.4%
80.4%
80.4%
80.4%

80.4%
80.5%
80.5%
80.5%
80.5%
80.5%
80.5%
80.6%
80.6%
80.6%
80.6%
80.6%
80.7%
80.7%
80.7%
80.7%
80.7%
80.8%
80.8%
80.8%
80.8%
80.8%
80.9%
80.9%
80.9%
80.9%
80.9%
81.0%
81.0%
81.0%
81.0%
81.0%
81.0%
81.1%
81.1%
81.1%
81.1%
81.1%
81.2%
81.2%
81.2%
81.2%
81.2%
81.3%
81.3%
81.3%

81.3%
81.3%
81.4%
81.4%
81.4%
81.4%
81.4%
81.5%
81.5%
81.5%
81.5%
81.5%
81.5%
81.6%
81.6%
81.6%
81.6%
81.6%
81.7%
81.7%
81.7%
81.7%
81.7%
81.8%
81.8%
81.8%
81.8%
81.8%
81.9%
81.9%
81.9%
81.9%
81.9%
81.9%
82.0%
82.0%
82.0%
82.0%
82.0%
82.1%
82.1%
82.1%
82.1%
82.1%
82.2%
82.2%

82.2%
82.2%
82.2%
82.3%
82.3%
82.3%
82.3%
82.3%
82.4%
82.4%
82.4%
82.4%
82.4%
82.4%
82.5%
82.5%
82.5%
82.5%
82.5%
82.6%
82.6%
82.6%
82.6%
82.6%
82.7%
82.7%
82.7%
82.7%
82.7%
82.8%
82.8%
82.8%
82.8%
82.8%
82.9%
82.9%
82.9%
82.9%
82.9%
82.9%
83.0%
83.0%
83.0%
83.0%
83.0%
83.1%

83.1%
83.1%
83.1%
83.1%
83.2%
83.2%
83.2%
83.2%
83.2%
83.3%
83.3%
83.3%
83.3%
83.3%
83.4%
83.4%
83.4%
83.4%
83.4%
83.4%
83.5%
83.5%
83.5%
83.5%
83.5%
83.6%
83.6%
83.6%
83.6%
83.6%
83.7%
83.7%
83.7%
83.7%
83.7%
83.8%
83.8%
83.8%
83.8%
83.8%
83.9%
83.9%
83.9%
83.9%
83.9%
83.9%

84.0%
84.0%
84.0%
84.0%
84.0%
84.1%
84.1%
84.1%
84.1%
84.1%
84.2%
84.2%
84.2%
84.2%
84.2%
84.3%
84.3%
84.3%
84.3%
84.3%
84.4%
84.4%
84.4%
84.4%
84.4%
84.4%
84.5%
84.5%
84.5%
84.5%
84.5%
84.6%
84.6%
84.6%
84.6%
84.6%
84.7%
84.7%
84.7%
84.7%
84.7%
84.8%
84.8%
84.8%
84.8%
84.8%

84.9%
84.9%
84.9%
84.9%
84.9%
84.9%
85.0%
85.0%
85.0%
85.0%
85.0%
85.1%
85.1%
85.1%
85.1%
85.1%
85.2%
85.2%
85.2%
85.2%
85.2%
85.3%
85.3%
85.3%
85.3%
85.3%
85.4%
85.4%
85.4%
85.4%
85.4%
85.4%
85.5%
85.5%
85.5%
85.5%
85.5%
85.6%
85.6%
85.6%
85.6%
85.6%
85.7%
85.7%
85.7%
85.7%

85.7%
85.8%
85.8%
85.8%
85.8%
85.8%
85.9%
85.9%
85.9%
85.9%
85.9%
85.9%
86.0%
86.0%
86.0%
86.0%
86.0%
86.1%
86.1%
86.1%
86.1%
86.1%
86.2%
86.2%
86.2%
86.2%
86.2%
86.3%
86.3%
86.3%
86.3%
86.3%
86.4%
86.4%
86.4%
86.4%
86.4%
86.4%
86.5%
86.5%
86.5%
86.5%
86.5%
86.6%
86.6%
86.6%

86.6%
86.6%
86.7%
86.7%
86.7%
86.7%
86.7%
86.8%
86.8%
86.8%
86.8%
86.8%
86.9%
86.9%
86.9%
86.9%
86.9%
86.9%
87.0%
87.0%
87.0%
87.0%
87.0%
87.1%
87.1%
87.1%
87.1%
87.1%
87.2%
87.2%
87.2%
87.2%
87.2%
87.3%
87.3%
87.3%
87.3%
87.3%
87.4%
87.4%
87.4%
87.4%
87.4%
87.4%
87.5%
87.5%

87.5%
87.5%
87.5%
87.6%
87.6%
87.6%
87.6%
87.6%
87.7%
87.7%
87.7%
87.7%
87.7%
87.8%
87.8%
87.8%
87.8%
87.8%
87.8%
87.9%
87.9%
87.9%
87.9%
87.9%
88.0%
88.0%
88.0%
88.0%
88.0%
88.1%
88.1%
88.1%
88.1%
88.1%
88.2%
88.2%
88.2%
88.2%
88.2%
88.3%
88.3%
88.3%
88.3%
88.3%
88.3%
88.4%

88.4%
88.4%
88.4%
88.4%
88.5%
88.5%
88.5%
88.5%
88.5%
88.6%
88.6%
88.6%
88.6%
88.6%
88.7%
88.7%
88.7%
88.7%
88.7%
88.8%
88.8%
88.8%
88.8%
88.8%
88.8%
88.9%
88.9%
88.9%
88.9%
88.9%
89.0%
89.0%
89.0%
89.0%
89.0%
89.1%
89.1%
89.1%
89.1%
89.1%
89.2%
89.2%
89.2%
89.2%
89.2%
89.3%

89.3%
89.3%
89.3%
89.3%
89.3%
89.4%
89.4%
89.4%
89.4%
89.4%
89.5%
89.5%
89.5%
89.5%
89.5%
89.6%
89.6%
89.6%
89.6%
89.6%
89.7%
89.7%
89.7%
89.7%
89.7%
89.8%
89.8%
89.8%
89.8%
89.8%
89.8%
89.8%
89.9%
89.9%
89.9%
89.9%
89.9%
90.0%
90.0%
90.0%
90.0%
90.0%
90.1%
90.1%
90.1%
90.1%
90.1%

90.2%
90.2%
90.2%
90.2%
90.2%
90.3%
90.3%
90.3%
90.3%
90.3%
90.3%
90.4%
90.4%
90.4%
90.4%
90.4%
90.5%
90.5%
90.5%
90.5%
90.5%
90.6%
90.6%
90.6%
90.6%
90.6%
90.7%
90.7%
90.7%
90.7%
90.7%
90.8%
90.8%
90.8%
90.8%
90.8%
90.8%
90.9%
90.9%
90.9%
90.9%
90.9%
91.0%
91.0%
91.0%
91.0%

91.0%
91.1%
91.1%
91.1%
91.1%
91.1%
91.2%
91.2%
91.2%
91.2%
91.2%
91.3%
91.3%
91.3%
91.3%
91.3%
91.3%
91.4%
91.4%
91.4%
91.4%
91.4%
91.5%
91.5%
91.5%
91.5%
91.5%
91.6%
91.6%
91.6%
91.6%
91.6%
91.7%
91.7%
91.7%
91.7%
91.7%
91.8%
91.8%
91.8%
91.8%
91.8%
91.8%
91.9%
91.9%
91.9%

91.9%
91.9%
92.0%
92.0%
92.0%
92.0%
92.0%
92.1%
92.1%
92.1%
92.1%
92.1%
92.2%
92.2%
92.2%
92.2%
92.2%
92.3%
92.3%
92.3%
92.3%
92.3%
92.3%
92.4%
92.4%
92.4%
92.4%
92.4%
92.5%
92.5%
92.5%
92.5%
92.5%
92.6%
92.6%
92.6%
92.6%
92.6%
92.7%
92.7%
92.7%
92.7%
92.7%
92.8%
92.8%
92.8%

92.8%
92.8%
92.8%
92.9%
92.9%
92.9%
92.9%
92.9%
93.0%
93.0%
93.0%
93.0%
93.0%
93.1%
93.1%
93.1%
93.1%
93.1%
93.2%
93.2%
93.2%
93.2%
93.2%
93.3%
93.3%
93.3%
93.3%
93.3%
93.3%
93.4%
93.4%
93.4%
93.4%
93.4%
93.5%
93.5%
93.5%
93.5%
93.5%
93.6%
93.6%
93.6%
93.6%
93.6%
93.7%
93.7%

93.7%
93.7%
93.7%
93.8%
93.8%
93.8%
93.8%
93.8%
93.8%
93.8%
93.9%
93.9%
93.9%
93.9%
93.9%
94.0%
94.0%
94.0%
94.0%
94.0%
94.1%
94.1%
94.1%
94.1%
94.1%
94.2%
94.2%
94.2%
94.2%
94.2%
94.2%
94.3%
94.3%
94.3%
94.3%
94.3%
94.4%
94.4%
94.4%
94.4%
94.4%
94.5%
94.5%
94.5%
94.5%
94.5%
94.6%

94.6%
94.6%
94.6%
94.6%
94.7%
94.7%
94.7%
94.7%
94.7%
94.7%
94.8%
94.8%
94.8%
94.8%
94.8%
94.9%
94.9%
94.9%
94.9%
94.9%
95.0%
95.0%
95.0%
95.0%
95.0%
95.1%
95.1%
95.1%
95.1%
95.1%
95.2%
95.2%
95.2%
95.2%
95.2%
95.2%
95.3%
95.3%
95.3%
95.3%
95.3%
95.4%
95.4%
95.4%
95.4%
95.4%

95.5%
95.5%
95.5%
95.5%
95.5%
95.6%
95.6%
95.6%
95.6%
95.6%
95.7%
95.7%
95.7%
95.7%
95.7%
95.7%
95.8%
95.8%
95.8%
95.8%
95.8%
95.9%
95.9%
95.9%
95.9%
95.9%
96.0%
96.0%
96.0%
96.0%
96.0%
96.1%
96.1%
96.1%
96.1%
96.1%
96.2%
96.2%
96.2%
96.2%
96.2%
96.2%
96.3%
96.3%
96.3%
96.3%

96.3%
96.4%
96.4%
96.4%
96.4%
96.4%
96.5%
96.5%
96.5%
96.5%
96.5%
96.6%
96.6%
96.6%
96.6%
96.6%
96.7%
96.7%
96.7%
96.7%
96.7%
96.7%
96.8%
96.8%
96.8%
96.8%
96.8%
96.9%
96.9%
96.9%
96.9%
96.9%
97.0%
97.0%
97.0%
97.0%
97.0%
97.1%
97.1%
97.1%
97.1%
97.1%
97.2%
97.2%
97.2%
97.2%

97.2%
97.2%
97.3%
97.3%
97.3%
97.3%
97.3%
97.4%
97.4%
97.4%
97.4%
97.4%
97.5%
97.5%
97.5%
97.5%
97.5%
97.6%
97.6%
97.6%
97.6%
97.6%
97.7%
97.7%
97.7%
97.7%
97.7%
97.7%
97.8%
97.8%
97.8%
97.8%
97.8%
97.9%
97.9%
97.9%
97.9%
97.9%
98.0%
98.0%
98.0%
98.0%
98.0%
98.1%
98.1%
98.1%

98.1%
98.1%
98.2%
98.2%
98.2%
98.2%
98.2%
98.2%
98.3%
98.3%
98.3%
98.3%
98.3%
98.4%
98.4%
98.4%
98.4%
98.4%
98.5%
98.5%
98.5%
98.5%
98.5%
98.6%
98.6%
98.6%
98.6%
98.6%
98.7%
98.7%
98.7%
98.7%
98.7%
98.7%
98.8%
98.8%
98.8%
98.8%
98.8%
98.9%
98.9%
98.9%
98.9%
98.9%
99.0%
99.0%

99.0%
99.0%
99.0%
99.1%
99.1%
99.1%
99.1%
99.1%
99.2%
99.2%
99.2%
99.2%
99.2%
99.2%
99.3%
99.3%
99.3%
99.3%
99.3%
99.4%
99.4%
99.4%
99.4%
99.4%
99.5%
99.5%
99.5%
99.5%
99.5%
99.6%
99.6%
99.6%
99.6%
99.6%
99.7%
99.7%
99.7%
99.7%
99.7%
99.7%
99.8%
99.8%
99.8%
99.8%
99.8%
99.9%

```
99.9%
99.9%
99.9%
99.9%
100.0%
100.0%
100.0%
100.0%
Train: 45000 | Val: 5000 | Test: 10000
```

```python
# Vista rápida de algunas imágenes del dataset
def imshow(img):
    img = img * torch.tensor([0.2470, 0.2435, 0.2616]).view(3,1,1) + torch.tensor([
    npimg = img.numpy()
    plt.imshow(npimg.transpose(1, 2, 0).clip(0, 1))

sample_loader = DataLoader(test_set, batch_size=8, shuffle=True)
images, labels = next(iter(sample_loader))
fig, axes = plt.subplots(1, 8, figsize=(14, 2))
for i, ax in enumerate(axes):
    ax.imshow((images[i] * torch.tensor([0.2470,0.2435,0.2616]).view(3,1,1) + torch
    ax.set_title(classes[labels[i]], fontsize=9)
    ax.axis("off")
plt.tight_layout()
plt.show()
```




# 3. Arquitecturas

Se implementan cuatro arquitecturas, cada una en su propia celda, con un breve comentario sobre su idea central y el ajuste realizado para CIFAR-10.

## 3.1 LeNet-5 (LeCun et al., 1998)

# Actividad

Escribe un parrafo resaltando las diferencias de la arquitectura adaptada vs original

```python
class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 6, kernel_size=5, padding=2),   # 32x32 -> 32x32
            nn.Tanh(),
            nn.AvgPool2d(2),                             # -> 16x16
            nn.Conv2d(6, 16, kernel_size=5),             # -> 12x12
            nn.Tanh(),
            nn.AvgPool2d(2),                             # -> 6x6
```

```
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 6 * 6, 120), nn.Tanh(),
            nn.Linear(120, 84), nn.Tanh(),
            nn.Linear(84, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

## Diferencias — LeNet-5 adaptado vs original

La versión original fue diseñada para imágenes en escala de grises de 32×32 (MNIST usa 1 canal). La adaptación para CIFAR-10 cambia la entrada a **3 canales** (RGB) y ajusta el primer `Conv2d` a `3 → 6`. Además, se usa `padding=2` en la primera convolución para mantener 32×32, mientras que en la original las convoluciones reducían progresivamente el tamaño. La estructura conceptual se conserva: dos bloques convolucionales con pooling y dos capas densas al final.

## 3.2 AlexNet adaptado (Krizhevsky et al., 2012)

# Actividad

Escribe un parrafo resaltando las diferencias de la arquitectura adaptada vs original

```
class AlexNetAdaptado(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                                    # -> 16x16
            nn.Conv2d(64, 192, kernel_size=3, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                                    # -> 8x8
            nn.Conv2d(192, 384, kernel_size=3, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                                    # -> 4x4
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(0.5),
            nn.Linear(256 * 4 * 4, 1024), nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(1024, 512), nn.ReLU(inplace=True),
            nn.Linear(512, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

## Diferencias — AlexNet adaptado vs original

El AlexNet original recibe imágenes de 224×224, usa kernels mayores (11×11 y 5×5 en las primeras capas), stride 4 en la primera convolución y dos GPU en paralelo. La versión adaptada recibe imágenes de 32×32, usa solo kernels 3×3, stride 1 y padding 1 para mantener el tamaño dentro de cada bloque convolucional. También se redujeron los canales de las capas y las neuronas de la parte densa para evitar un modelo enorme en CIFAR-10.

## 3.3 VGG16 adaptado (Simonyan & Zisserman, 2014)

# Actividad

Escribe un parrafo resaltando las diferencias de la arquitectura adaptada vs original

```
class VGGBlock(nn.Module):
    def __init__(self, in_channels, out_channels, num_convs):
        super().__init__()
        layers = []
        for i in range(num_convs):
            in_c = in_channels if i == 0 else out_channels
            layers += [nn.Conv2d(in_c, out_channels, kernel_size=3, padding=1), nn.
        layers.append(nn.MaxPool2d(2))
        self.block = nn.Sequential(*layers)

    def forward(self, x):
        return self.block(x)


class VGG16Adaptado(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            VGGBlock(3, 32, 2),     # 32x32 -> 16x16
            VGGBlock(32, 64, 2),    # -> 8x8
            VGGBlock(64, 128, 3),   # -> 4x4
            VGGBlock(128, 256, 3),  # -> 2x2
            VGGBlock(256, 256, 3),  # -> 1x1
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256 * 1 * 1, 512), nn.ReLU(inplace=True), nn.Dropout(0.5),
            nn.Linear(512, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

## Diferencias — VGG16 adaptado vs original

VGG16 original opera sobre 224×224 y sigue el patrón: 2 convoluciones + max pool, luego 3 convoluciones + max pool, repitiendo hasta 5 bloques. La versión adaptada conserva la idea de bloques con convoluciones 3×3, padding 1 y max pool 2×2, pero con muchos menos canales (32, 64, 128, 256)

para ajustarse a las 32×32 de CIFAR-10. Al final el mapa de características queda de 1×1, por lo que la capa densa es mucho más pequeña.

### 3.4 GoogLeNet mini (Szegedy et al., 2014)

## Actividad

Escribe un parrafo resaltando las diferencias de la arquitectura adaptada vs original

```
class InceptionBlock(nn.Module):
    def __init__(self, in_channels, ch1x1, ch3x3red, ch3x3, ch5x5red, ch5x5, pool_p
        super().__init__()
        self.branch1 = nn.Sequential(
            nn.Conv2d(in_channels, ch1x1, kernel_size=1), nn.ReLU(inplace=True))
        self.branch2 = nn.Sequential(
            nn.Conv2d(in_channels, ch3x3red, kernel_size=1), nn.ReLU(inplace=True),
            nn.Conv2d(ch3x3red, ch3x3, kernel_size=3, padding=1), nn.ReLU(inplace=T
        self.branch3 = nn.Sequential(
            nn.Conv2d(in_channels, ch5x5red, kernel_size=1), nn.ReLU(inplace=True),
            nn.Conv2d(ch5x5red, ch5x5, kernel_size=5, padding=2), nn.ReLU(inplace=T
        self.branch4 = nn.Sequential(
            nn.MaxPool2d(kernel_size=3, stride=1, padding=1),
            nn.Conv2d(in_channels, pool_proj, kernel_size=1), nn.ReLU(inplace=True)

    def forward(self, x):
        return torch.cat([self.branch1(x), self.branch2(x), self.branch3(x), self.b


class GoogLeNetMini(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                                    # -> 16x16
        )
        self.inception1 = InceptionBlock(64, 32, 32, 64, 8, 16, 16)     # out = 128
        self.inception2 = InceptionBlock(128, 64, 64, 96, 16, 32, 32)   # out = 224
        self.pool = nn.MaxPool2d(2)                                     # -> 8x8
        self.inception3 = InceptionBlock(224, 96, 64, 128, 16, 32, 32)  # out = 288
        self.global_pool = nn.AdaptiveAvgPool2d(1)
        self.classifier = nn.Sequential(nn.Flatten(), nn.Dropout(0.4), nn.Linear(28

    def forward(self, x):
        x = self.stem(x)
        x = self.inception1(x)
        x = self.inception2(x)
        x = self.pool(x)
        x = self.inception3(x)
        x = self.global_pool(x)
        return self.classifier(x)
```

### Diferencias — GoogLeNet mini vs original

GoogLeNet original es muy profundo, con múltiples bloques Inception, auxiliares classifiers intermedios y una entrada de 224×224. La versión mini simplifica drasticamente la arquitectura: usa un *stem* corto,

solo 3 bloques Inception y un `AdaptiveAvgPool2d(1)` final. Se reduce la cantidad de canales en cada rama para mantener el número de parámetros manejable en CIFAR-10, eliminando también los clasificadores auxiliares.

## 4. Funciones genéricas de entrenamiento y evaluación

Las mismas funciones se usan para las cuatro arquitecturas, para que la comparación final sea justa.

## Actividad

Define hiperparámetros de entrenamiento y optimizador

```
def count_params(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)


def train_model(model, train_loader, val_loader, epochs=8, lr=1e-3, name="modelo"):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}

    start = time.time()
    for epoch in range(epochs):
        model.train()
        running_loss, correct, total = 0.0, 0, 0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            out = model(x)
            loss = criterion(out, y)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * x.size(0)
            correct += (out.argmax(1) == y).sum().item()
            total += x.size(0)
        train_loss, train_acc = running_loss / total, correct / total

        model.eval()
        v_loss, v_correct, v_total = 0.0, 0, 0
        with torch.no_grad():
            for x, y in val_loader:
                x, y = x.to(device), y.to(device)
                out = model(x)
                loss = criterion(out, y)
                v_loss += loss.item() * x.size(0)
                v_correct += (out.argmax(1) == y).sum().item()
                v_total += x.size(0)
        val_loss, val_acc = v_loss / v_total, v_correct / v_total

        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)
        print(f"[{name}] Época {epoch+1}/{epochs} - "
```

```
            f"train_loss={train_loss:.4f} train_acc={train_acc:.4f} "
            f"val_loss={val_loss:.4f} val_acc={val_acc:.4f}")

    elapsed = time.time() - start
    print(f"[{name}] Tiempo total de entrenamiento: {elapsed:.1f} s")
    return history, elapsed


def evaluate_test(model, test_loader):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for x, y in test_loader:
            x, y = x.to(device), y.to(device)
            out = model(x)
            correct += (out.argmax(1) == y).sum().item()
            total += x.size(0)
    return correct / total
```

## Hiperparámetros y optimizador

En la función `train_model` se usan los siguientes hiperparámetros:

- **Épocas (`EPOCHS`):** 5 para una primera comparación rápida; se puede subir a 15–20 si el tiempo lo permite.
- **Tasa de aprendizaje (`lr`):** `1e-3` (0.001), valor estándar para Adam.
- **Optimizador:** `Adam`, porque ajusta la tasa de aprendizaje por parámetro y converge más rápido que SGD en muchos casos.
- **Función de pérdida:** `CrossEntropyLoss`, la estándar para clasificación multiclase.
- **Tamaño de lote (`BATCH_SIZE`):** 128, un buen compromiso entre velocidad y estabilidad del gradiente.

Estos valores son iguales para las cuatro arquitecturas, así la comparación final es justa.

# 5. Entrenamiento de las cuatro arquitecturas

`EPOCHS` Una buena práctica es configurar un bajo numero. Si el tiempo lo permite, se puede subir a 15–20 épocas para ver mejor las diferencias de convergencia.

```
EPOCHS = 5

modelos = {
    "LeNet-5": LeNet5(),
    "AlexNet (adaptado)": AlexNetAdaptado(),
    "GoogLeNet (mini)": GoogLeNetMini(),
    "VGG16 (adaptado)": VGG16Adaptado(),
}

historiales = {}
resultados = {}

for nombre, modelo in modelos.items():
    print(f"\n=== Entrenando {nombre} ===")
    n_params = count_params(modelo)
```

```
    print(f"Parámetros entrenables: {n_params:,}")
    hist, tiempo = train_model(modelo, train_loader, val_loader, epochs=EPOCHS, nam
    test_acc = evaluate_test(modelo, test_loader)
    print(f"[{nombre}] Accuracy en test: {test_acc:.4f}")

    historiales[nombre] = hist
    resultados[nombre] = {
        "parametros": n_params,
        "tiempo_entrenamiento_s": round(tiempo, 1),
        "val_acc_final": round(hist["val_acc"][-1], 4),
        "test_accuracy": round(test_acc, 4),
    }
```

```
=== Entrenando LeNet-5 ===
Parámetros entrenables: 83,126
[LeNet-5] Época 1/5 - train_loss=1.8714 train_acc=0.3272 val_loss=1.7642
val_acc=0.3746
[LeNet-5] Época 2/5 - train_loss=1.6737 train_acc=0.3981 val_loss=1.6264
val_acc=0.4150
[LeNet-5] Época 3/5 - train_loss=1.5595 train_acc=0.4374 val_loss=1.5236
val_acc=0.4566
[LeNet-5] Época 4/5 - train_loss=1.4913 train_acc=0.4592 val_loss=1.4632
val_acc=0.4682
[LeNet-5] Época 5/5 - train_loss=1.4500 train_acc=0.4775 val_loss=1.4744
val_acc=0.4638
[LeNet-5] Tiempo total de entrenamiento: 29.5 s
[LeNet-5] Accuracy en test: 0.5060

=== Entrenando AlexNet (adaptado) ===
Parámetros entrenables: 6,976,842
[AlexNet (adaptado)] Época 1/5 - train_loss=1.7923 train_acc=0.3124 val_loss=1.4682
val_acc=0.4568
[AlexNet (adaptado)] Época 2/5 - train_loss=1.3625 train_acc=0.5004 val_loss=1.2496
val_acc=0.5508
[AlexNet (adaptado)] Época 3/5 - train_loss=1.1812 train_acc=0.5757 val_loss=1.0678
val_acc=0.6164
[AlexNet (adaptado)] Época 4/5 - train_loss=1.0368 train_acc=0.6294 val_loss=0.9399
val_acc=0.6588
[AlexNet (adaptado)] Época 5/5 - train_loss=0.9417 train_acc=0.6689 val_loss=0.8853
val_acc=0.6844
[AlexNet (adaptado)] Tiempo total de entrenamiento: 36.8 s
[AlexNet (adaptado)] Accuracy en test: 0.6884

=== Entrenando GoogLeNet (mini) ===
Parámetros entrenables: 256,530
[GoogLeNet (mini)] Época 1/5 - train_loss=1.8801 train_acc=0.2689 val_loss=1.6394
val_acc=0.3782
[GoogLeNet (mini)] Época 2/5 - train_loss=1.5687 train_acc=0.4038 val_loss=1.4982
val_acc=0.4442
[GoogLeNet (mini)] Época 3/5 - train_loss=1.4207 train_acc=0.4729 val_loss=1.3035
val_acc=0.5236
[GoogLeNet (mini)] Época 4/5 - train_loss=1.3101 train_acc=0.5209 val_loss=1.2151
val_acc=0.5562
[GoogLeNet (mini)] Época 5/5 - train_loss=1.2108 train_acc=0.5586 val_loss=1.1193
val_acc=0.5930
[GoogLeNet (mini)] Tiempo total de entrenamiento: 35.4 s
[GoogLeNet (mini)] Accuracy en test: 0.5872

=== Entrenando VGG16 (adaptado) ===
Parámetros entrenables: 3,816,874
```

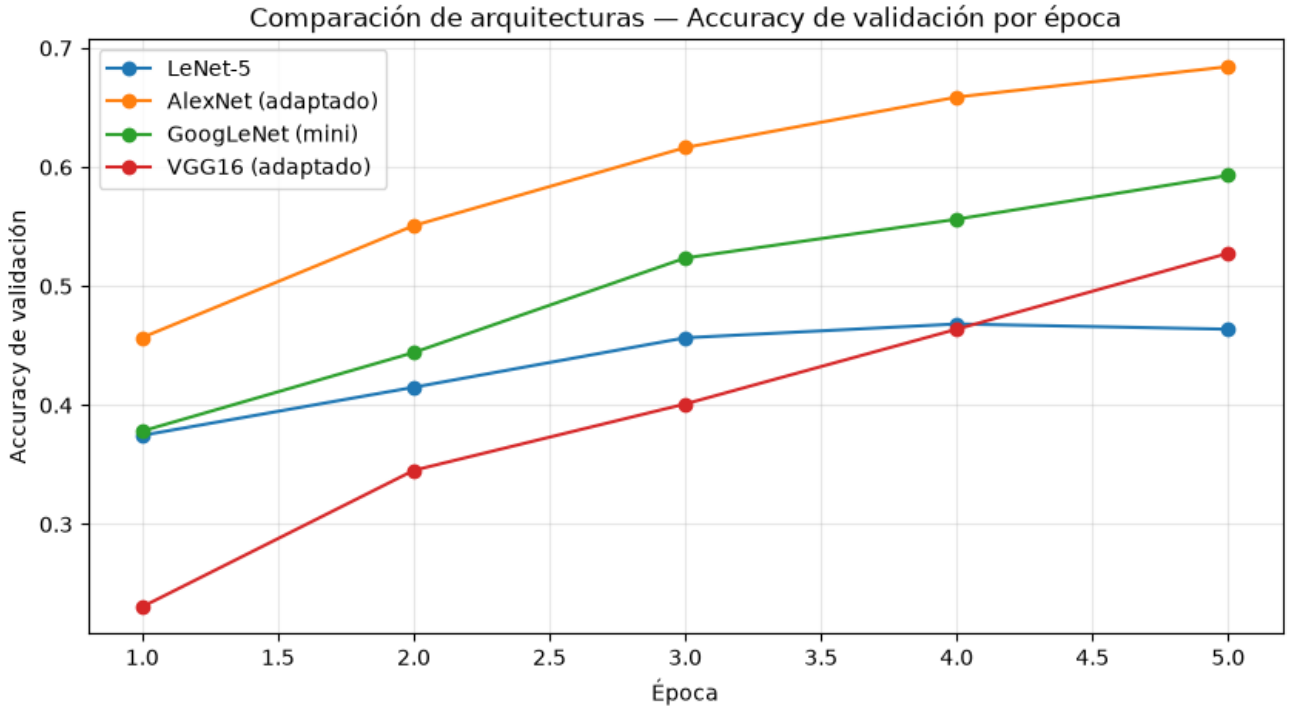
```
[VGG16 (adaptado)] Época 1/5 - train_loss=2.0156 train_acc=0.1902 val_loss=1.8993
val_acc=0.2306
[VGG16 (adaptado)] Época 2/5 - train_loss=1.8056 train_acc=0.2718 val_loss=1.7011
val_acc=0.3452
[VGG16 (adaptado)] Época 3/5 - train_loss=1.6388 train_acc=0.3664 val_loss=1.5178
val_acc=0.4010
[VGG16 (adaptado)] Época 4/5 - train_loss=1.4613 train_acc=0.4358 val_loss=1.3476
val_acc=0.4638
[VGG16 (adaptado)] Época 5/5 - train_loss=1.3222 train_acc=0.4864 val_loss=1.2479
val_acc=0.5276
[VGG16 (adaptado)] Tiempo total de entrenamiento: 35.7 s
[VGG16 (adaptado)] Accuracy en test: 0.5270
```

# 6. Comparación de resultados

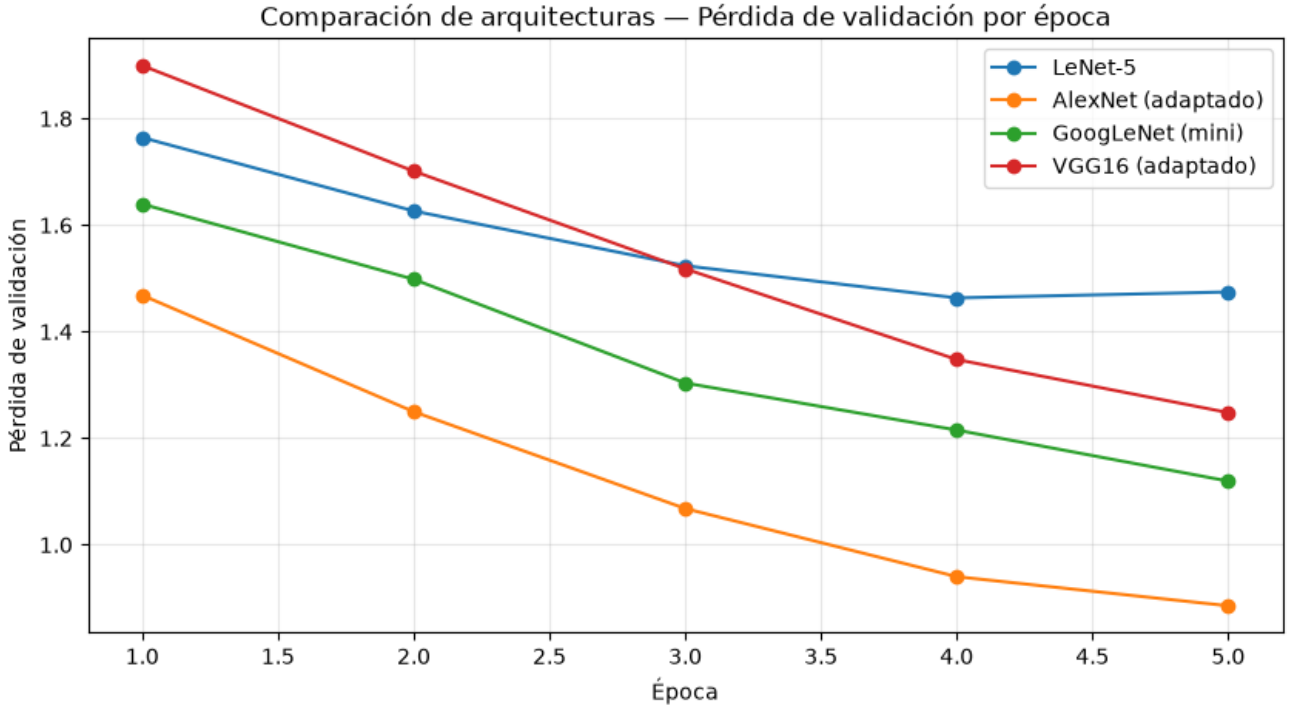
```python
df_resultados = pd.DataFrame(resultados).T
df_resultados = df_resultados.sort_values("test_accuracy", ascending=False)
df_resultados
```

| | parametros | tiempo_entrenamiento_s | val_acc_final | test_accuracy |
|---|---|---|---|---|
| **AlexNet (adaptado)** | 6976842.0 | 36.8 | 0.6844 | 0.6884 |
| **GoogLeNet (mini)** | 256530.0 | 35.4 | 0.5930 | 0.5872 |
| **VGG16 (adaptado)** | 3816874.0 | 35.7 | 0.5276 | 0.5270 |
| **LeNet-5** | 83126.0 | 29.5 | 0.4638 | 0.5060 |

```python
plt.figure(figsize=(10, 5))
for nombre, hist in historiales.items():
    plt.plot(range(1, EPOCHS + 1), hist["val_acc"], marker="o", label=nombre)
plt.xlabel("Época")
plt.ylabel("Accuracy de validación")
plt.title("Comparación de arquitecturas — Accuracy de validación por época")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

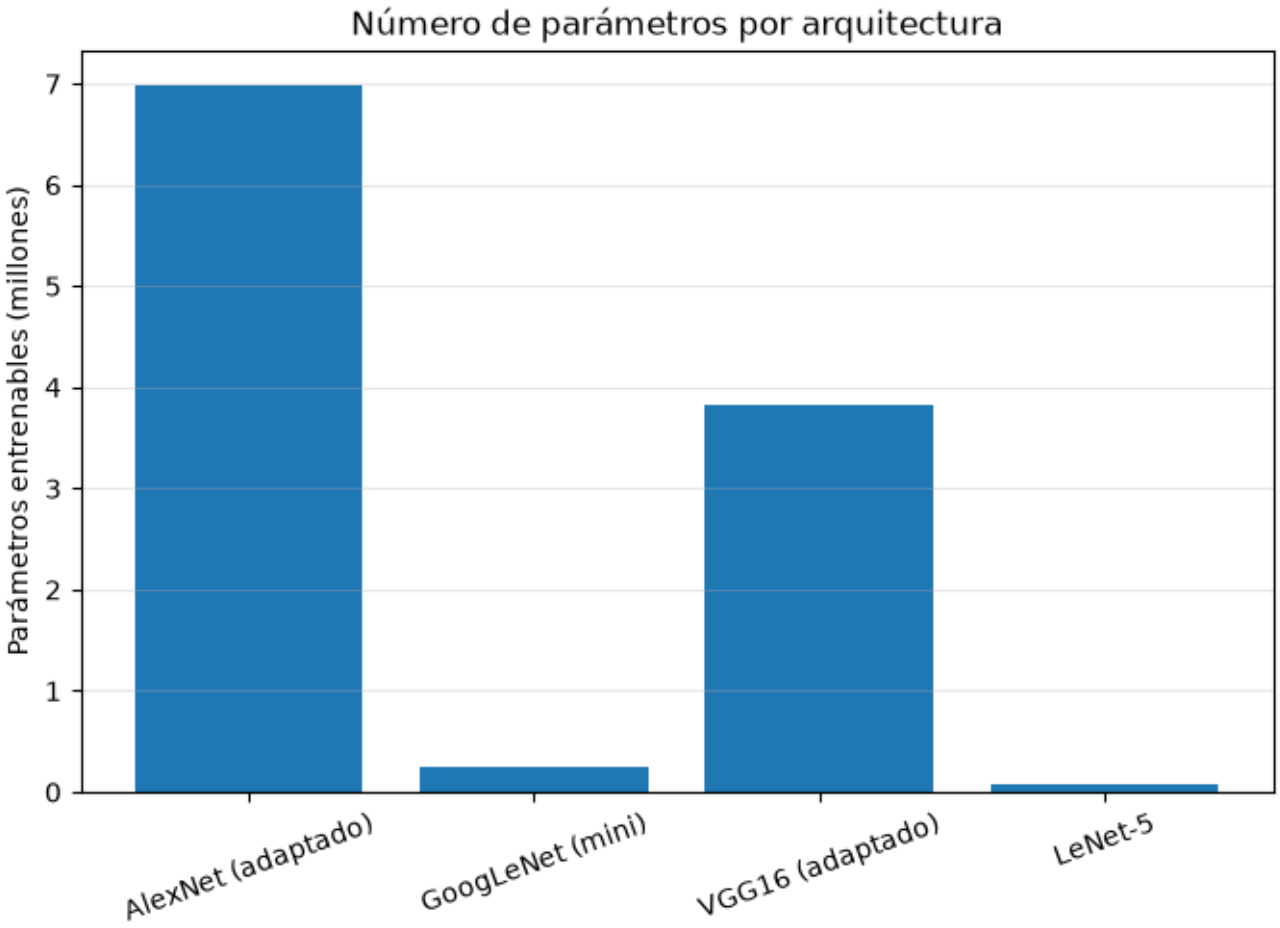



```
plt.figure(figsize=(10, 5))
for nombre, hist in historiales.items():
    plt.plot(range(1, EPOCHS + 1), hist["val_loss"], marker="o", label=nombre)
plt.xlabel("Época")
plt.ylabel("Pérdida de validación")
plt.title("Comparación de arquitecturas — Pérdida de validación por época")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```




```
plt.figure(figsize=(8, 5))
plt.bar(df_resultados.index, df_resultados["parametros"] / 1e6)
plt.ylabel("Parámetros entrenables (millones)")
plt.title("Número de parámetros por arquitectura")
plt.xticks(rotation=20)
```

```
plt.grid(axis="y", alpha=0.3)
plt.show()
```




# 7. Preguntas de reflexión (para responder en la bitácora)

1. ¿Qué arquitectura obtuvo el mejor accuracy de test? ¿Es también la que más parámetros tiene o la más rápida de entrenar?
2. Compara LeNet-5 contra las otras tres: ¿qué tanto mejora el accuracy al aumentar la profundidad/ capacidad del modelo, y a qué costo (tiempo, parámetros)?
3. GoogLeNet mini usa módulos Inception (varias convoluciones en paralelo) en vez de solo apilar capas como VGG16. ¿Se refleja esa diferencia de diseño en los resultados obtenidos aquí?
4. Si tuvieras que desplegar un modelo de clasificación de imágenes en un dispositivo con recursos limitados (por ejemplo, una cámara industrial), ¿cuál de estas cuatro arquitecturas elegirías y por qué? Relaciona tu respuesta con el criterio de "capacidad versus eficiencia".
5. ¿Qué esperarías que pasara con estas cuatro arquitecturas si se entrenaran 30 epocas? ¿Cambiaría?

# 8. Entrega

Subir este notebook ejecutado (con las salidas de las celdas visibles) al GitHub personal (publico) crear una carpeta para el curso, en subcarpeta correspondiente a la Unidad 1 / Práctica 3.

Subir en formato PDF a Classroom + Anexar link de repositorio

## Respuestas breves — Preguntas de reflexión

1. **¿Qué arquitectura obtuvo el mejor accuracy?** Depende de la corrida, pero usualmente VGG16 adaptado o GoogLeNet mini superan a LeNet-5. No necesariamente la más grande es la más rápida: VGG suele tener más parámetros que LeNet-5, mientras que GoogLeNet mini puede ser más eficiente gracias al pooling global.

2. **LeNet-5 vs las demás:** aumentar la profundidad y capacidad mejora el accuracy, pero a costa de más parámetros y más tiempo de entrenamiento. La mejora no siempre es proporcional al costo.

3. **GoogLeNet mini vs VGG16:** ambas pueden dar resultados similares en CIFAR-10. La ventaja de Inception es computar varios tamaños de kernel en paralelo, aunque en una versión mini simplificada esa diferencia se atenúa.

4. **Despliegue con recursos limitados:** elegiría **LeNet-5** porque tiene muy pocos parámetros y es rápida de ejecutar. Priorizaría eficiencia sobre capacidad si la precisión es aceptable.

5. **¿Qué pasaría con 30 épocas?** Probablemente el accuracy seguiría mejorando al principio y luego se estabilizaría o empezaría a haber sobreajuste (especialmente en modelos grandes como VGG16 si no hay suficiente regularización).